

UNIVERSITY OF HERTFORDSHIRE

Department of Computer Science

Modular BSc Honours in Computer Science

6COM2019 - Cyber Security and Networks Project

Final Report

March 2025

TITLE OF PROJECT

*KeyWatch: Keystroke Behaviour Monitoring & Anomaly Detection
Dashboard*

Author's initials and surname

Samarth Desai

Supervised by:

Azita Pourghasem

Abstract

The increasing sophistication of cyber threats necessitates innovative solutions that balance real-time anomaly detection with stringent privacy compliance. This project introduces KeyWatch, a GDPR-compliant system for monitoring keystroke dynamics and identifying behavioural anomalies through unsupervised machine learning. Developed using `Python`, `Dash`, and `scikit-learn`, KeyWatch integrates a local keylogger, an Isolation Forest-based detection model, and an interactive dashboard. Agile methodologies guided the development process, with iterative stakeholder feedback refining system functionality.

The system captures keystroke dwell and flight times while explicitly excluding sensitive content, aligning with GDPR's data minimisation (Article 5) and privacy-by-design (Article 25) requirements. Isolation Forest was selected for its efficiency in handling unlabelled, high-dimensional data, achieving 92% precision and 89% recall in anomaly detection. The dashboard offers real-time visualisations (updated every 15 seconds), adjustable sensitivity controls, and resource monitoring—processed entirely locally to ensure full data sovereignty.

Technical challenges, including data synchronisation latency and model overfitting, were mitigated through `MinMaxScaler` normalisation, scheduled model retraining, and `Dash`'s `dcc.Store` caching mechanism. Testing confirmed robust performance on consumer-grade hardware, with average CPU usage at 33.35% and latency consistently under 12 seconds.

KeyWatch's core contribution lies in its ability to mitigate risks such as session hijacking and insider threats through privacy-first behavioural biometrics. Future iterations may incorporate federated learning to enable multi-user support and real-time alerting. This work demonstrates the practical feasibility of ethical, machine learning-driven security solutions deployable within GDPR-regulated environments.

Keywords: Keystroke dynamics, Isolation Forest, GDPR compliance, anomaly detection, real-time dashboard, behavioural biometrics.

Acknowledgements

My sincere thanks to my supervisor, Azita Pourghasem, for her invaluable guidance, continuous support, and constructive feedback throughout the entire duration of this project. Her expertise and encouragement have significantly contributed to the successful completion of my work.

Special thanks to my lecturer, Paul Morris, whose insightful teaching and practical advice equipped me with the foundational knowledge and skills essential for this project's implementation.

Special thanks are extended to our module leader, Pusp Joshi, for his exceptional coordination, clear communication, and support, ensuring the smooth progression of the module and addressing all concerns promptly.

Lastly, my heartfelt appreciation goes to my family and friends for their unwavering support, patience, and encouragement throughout this journey. Their belief in me provided the motivation and strength required to complete this demanding yet rewarding project.

Table of Contents

1	INTRODUCTION.....	1
1.1	PROJECT CONTEXT AND BACKGROUND.....	1
1.2	AIM AND OBJECTIVES	2
1.3	SIGNIFICANCE IN CYBERSECURITY	3
1.4	METHODOLOGY OVERVIEW	3
1.5	REPORT STRUCTURE	3
2	LITERATURE REVIEW	5
2.1	KEYSTROKE DYNAMICS AND BEHAVIOURAL BIOMETRICS	5
2.2	MACHINE LEARNING TECHNIQUES FOR ANOMALY DETECTION.....	5
2.2.1	ISOLATION FOREST	5
2.2.2	ONE-CLASS SVM AND AUTOENCODERS.....	6
2.3	GDPR COMPLIANCE AND PRIVACY BY DESIGN.....	7
2.4	IDENTIFIED GAPS AND JUSTIFICATION OF <i>KEYWATCH</i>.....	7
2.5	PRACTICAL CYBERSECURITY APPLICATIONS OF KEYSTROKE DYNAMICS.....	7
2.6	SUMMARY AND RECOMMENDATIONS.....	8
3	METHODOLOGY AND DESIGN.....	9
3.1	CHAPTER OVERVIEW	9
3.2	PROJECT METHODOLOGY	9
3.2.1	AGILE DEVELOPMENT PHASES.....	9
3.3	SYSTEM DESIGN APPROACH.....	10
3.3.1	BACKEND DESIGN.....	10
3.3.2	FRONTEND DESIGN.....	13
3.4	TOOLS AND TECHNOLOGIES	15
3.5	COMPLIANCE AND ETHICAL CONSIDERATIONS.....	16
3.5.1	GDPR COMPLIANCE	16
3.5.2	ETHICAL CONSIDERATIONS	17
3.6	RISK MANAGEMENT.....	17
3.6.1	IDENTIFICATION AND MITIGATION STRATEGIES	17
3.6.2	CONTINUOUS RISK MONITORING.....	18
3.7	PROFESSIONAL AND ETHICAL RESPONSIBILITIES.....	18
3.7.1	PROFESSIONAL CONDUCT.....	18

3.7.2	ETHICAL RESPONSIBILITIES.....	18
3.7.3	ACADEMIC INTEGRITY AND INTELLECTUAL PROPERTY.....	19
4	IMPLEMENTATION.....	20
4.1	CHAPTER OVERVIEW	20
4.2	BACKEND IMPLEMENTATION	20
4.2.1	KEYLOGGER MODULE.....	20
4.2.2	ANOMALY DETECTION WITH ISOLATION FOREST.....	21
4.3	FRONTEND IMPLEMENTATION	22
4.3.1	DASHBOARD ARCHITECTURE.....	22
4.3.2	RESOURCE MONITORING SYSTEM.....	23
4.4	INTEGRATION CHALLENGES	24
4.4.1	DATA SYNCHRONISATION.....	25
4.4.2	PERFORMANCE OPTIMISATION.....	25
4.4.3	DEBUGGING AND ERROR HANDLING.....	26
5	TESTING AND VALIDATION	27
5.1	CHAPTER OVERVIEW	27
5.2	UNIT TESTING	27
5.3	FUNCTIONAL AND SYSTEM TESTING.....	28
5.4	RESOURCE USAGE TESTING.....	29
5.5	DEBUG LOG DIAGNOSTICS	30
5.6	ETHICAL AND PROFESSIONAL COMPLIANCE	31
5.7	SUMMARY OF TESTING OUTCOMES.....	31
5.8	LIMITATIONS AND FUTURE WORK.....	32
5.8.1	IDENTIFIED LIMITATION	32
5.8.2	FUTURE WORK AND ENHANCEMENTS.....	33
6	PROJECT EVALUATION.....	35
6.1	OVERVIEW	35
6.2	PERSONAL EVALUATION.....	35
6.3	RESEARCH EVALUATION	36
6.4	METHODOLOGY EVALUATION	37
6.5	ARTEFACT EVALUATION (AGAINST THE REQUIREMENTS)	37
7	CONCLUSION.....	39

8	REFERENCES.....	41
9	APPENDICES	43
9.1	APPENDIX A: PROJECT GANTT CHART	43
9.2	APPENDIX B: SYSTEM DESIGN DIAGRAMS	43
9.3	APPENDIX C: FRONTEND WIREFRAMES	44
9.4	APPENDIX D: SOURCE CODE SNIPPETS	45
9.5	APPENDIX E: DEBUG LOGS AND RESOURCE MONITORING DATA	58
9.6	APPENDIX F: KEYSTROKE DATA SAMPLE.....	59
9.7	APPENDIX G: THIRD-PARTY LIBRARIES AND LICENSES	60
9.8	APPENDIX H: INSTALLATION AND USER GUIDE	60

Table of Tables

TABLE 2.1: COMPARATIVE ANALYSIS OF ANOMALY DETECTION ALGORITHMS.....	6
TABLE 3.1: AGILE DEVELOPMENT PHASES.....	10
TABLE 3.2: TOOLS AND TECHNOLOGIES.....	15
TABLE 3.3: DETAILED PROJECT RISK, MITIGATION, AND OUTCOMES.....	18
TABLE 5.1: TESTING OUTCOMES.....	28
TABLE 5.2: FUNCTIONALITIES TABLE	29

Table of Figures

FIGURE 3.1: KEYWATCH USE CASE DIAGRAM.....	10
FIGURE 3.2: IMPLEMENTING MINMAXSCALER AND ISOLATIONFOREST	11
FIGURE 3.3: KEYWATCH SYSTEM PROCESS FLOWCHART.....	12
FIGURE 3.4: REAL-TIME DASHBOARD	13
FIGURE 3.5: DROPDOWN MENU FOR SENSITIVITY CONTROL.....	14
FIGURE 3. 6: LOCAL DATA HANDLING	15
FIGURE 4.1: KEYLOGGER’S EVENT HANDLING LOGIC.....	21
FIGURE 4.2: DASHBOARD LAYOUT	23
FIGURE 4.3: RESOURCE MONITOR LOG	24

1 Introduction

The increasing complexity and frequency of cybersecurity threats emphasise the critical need for innovative solutions capable of detecting and mitigating risks in real time. This project, titled "KeyWatch: Keystroke Behaviour Monitoring & Anomaly Detection Dashboard, " aims to address existing gaps in cybersecurity defences by combining keystroke logging, behavioural biometrics, and unsupervised machine learning within a locally processed, GDPR-compliant framework.

1.1 Project Context and Background

Cybersecurity threats have significantly evolved, with attackers frequently deploying covert keyloggers to intercept sensitive credentials and confidential information (Ali et al., 2022). Traditional defences, predominantly signature-based antivirus systems, struggle against zero-day attacks, necessitating more proactive and dynamic detection mechanisms. Behavioural biometrics, particularly keystroke dynamics, offer a compelling solution. Metrics such as dwell time (key hold duration) and flight time (interval between keystrokes) provide unique, difficult-to-replicate biometric signatures, significantly enhancing authentication accuracy (Sharma et al., 2023).

Currently, available anomaly detection solutions commonly leverage machine learning algorithms; however, they often rely on cloud infrastructure. This cloud dependency introduces significant privacy concerns under regulations such as the General Data Protection Regulation (GDPR), which mandates stringent data protection standards for EU citizens, requiring solutions to minimise data exposure ([EU, 2018](#)). Consequently, there is an evident need for open-source, locally processed, and privacy-focused anomaly detection tools suitable for organisational deployment.

KeyWatch directly addresses these requirements by ensuring local data processing without cloud dependency, adhering strictly to GDPR compliance. The Isolation Forest algorithm was selected as the anomaly detection method due to its computational efficiency, robustness in handling complex, high-dimensional datasets, and its ability to function without predefined thresholds or labelled datasets, making it highly suitable for unsupervised real-time anomaly detection (Liu et al., 2008).

1.2 Aim and Objectives

The primary aim of this project was to design and implement a comprehensive keystroke behaviour analytics system that captures typing patterns, detects anomalies using unsupervised machine learning, and provides an interactive dashboard for real-time visualisation and analysis.

The following SMART objectives were established and achieved during the project lifecycle:

- Objective 1: Complete a comprehensive literature review on keystroke logging, behavioural biometrics, and anomaly detection algorithms (Isolation Forest, One-Class SVM, and Autoencoders), identifying best practices for privacy-compliant local data processing.
- Objective 2: Produce a detailed system specification clearly defining functional and non-functional requirements, accompanied by flowcharts and data-flow diagrams illustrating keystroke data capture, anomaly detection processes, and dashboard visualisation methods.
- Objective 3: Design and document the backend architecture, including precise data structures, log formats, local storage methods, and efficient data handling processes to ensure optimal local performance.
- Objective 4: Design an intuitive and interactive frontend dashboard interface, including real-time keystroke visualisation, anomaly detection alerts, adjustable model sensitivity controls, and integrated resource usage monitoring.
- Objective 5: Develop and integrate backend and frontend components using Python, scikit-learn (Isolation Forest), Dash, and Plotly, maintaining local data processing to meet GDPR standards.
- Objective 6: Deliver a fully functional prototype for initial evaluation and iterative refinement based on feedback from the project supervisor and stakeholders.
- Objective 7: Conduct rigorous testing and performance optimisation, addressing identified issues to enhance anomaly detection accuracy, system reliability, and overall usability.
- Objective 8: Prepare and submit a comprehensive Final Project Report documenting the entire project lifecycle, including critical evaluations, reflections, and detailed appendices to demonstrate the project's success clearly and professionally.

1.3 Significance in Cybersecurity

KeyWatch's principal contribution to cybersecurity lies in its privacy-focused design, enabling organisations to leverage behavioural biometrics without compromising compliance. Its capability to perform real-time anomaly detection delivers substantial practical benefits; for instance, KeyWatch effectively mitigates risks such as session hijacking in financial systems, where attackers using compromised credentials exhibit significantly different typing patterns compared to legitimate users. Furthermore, the behavioural analytics capabilities of KeyWatch provide valuable insights relevant to essential cybersecurity frameworks, including Security Information and Event Management (SIEM), Intrusion Detection Systems (IDS), and User and Entity Behaviour Analytics (UEBA), significantly enhancing an organisation's overall security posture (Bhuyan et al., 2020)

1.4 Methodology Overview

This project adopted an Agile methodology, characterised by iterative development cycles implemented in two-week sprints. Regular feedback sessions with the supervisor were integral to these sprints, ensuring ongoing alignment with ethical guidelines, project specifications, and technical best practices, such as fine-tuning the Isolation Forest model's sensitivity parameters. Project milestones were carefully tracked and managed using a detailed Gantt chart (see Appendix A). This facilitated timely progress on critical tasks, including anomaly detection accuracy optimisation, data preprocessing techniques, and dashboard performance improvements. Technical challenges, notably the presence of noise and variability in dwell and flight time data, were systematically addressed using techniques like MinMaxScaler normalisation and periodic model retraining, resulting in a significant improvement in anomaly detection accuracy and reliability (further details in Section 4.2).

1.5 Report Structure

This report structure follows the guidelines provided by the University of Hertfordshire (2024), documenting each stage of the project's lifecycle logically and progressively. The chapters are organised as follows:

- Chapter 1: Introduction
 - Establishes the context and motivation behind the project, outlining its aims, objectives, significance, and a brief overview of the adopted methodology.

- Chapter 2: Literature Review
 - Critically evaluates existing research, identifying technological gaps and providing justification for the selected approaches within the KeyWatch system.
- Chapter 3: Methodology and Design
 - Describes the methodological framework, including Agile practices, system specifications, and architectural design. This chapter also includes relevant diagrams and models to support the design rationale.
- Chapter 4: Implementation
 - Presents the practical development process, detailing the backend and frontend implementation, integration strategies, and key technical decisions.
- Chapter 5: Testing and Validation
 - Explains the testing methodology and tools employed, accompanied by a detailed analysis of results and system performance. It also highlights limitations and suggests future enhancements.
- Chapter 6: Project Evaluation
 - Provides a critical reflection on the overall project, including the evaluation of objectives, personal performance, challenges faced, and lessons learned.
- Chapter 7: Conclusion
 - Summarises the key outcomes of the project, reinforcing its contributions and outlining directions for further development.

2 Literature Review

This literature review critically analyses existing research and technologies concerning keystroke dynamics, behavioural biometrics, anomaly detection, and machine learning methodologies. It establishes the theoretical foundation that informs the technical decisions and implementation strategies underpinning the *KeyWatch* project.

2.1 Keystroke Dynamics and Behavioural Biometrics

Keystroke dynamics, a specific category within behavioural biometrics, involves analysing users' typing rhythms, particularly dwell time—the duration a key is pressed—and flight time, the interval between keystrokes (Sharma et al., 2023). These behavioural metrics form distinct, individual biometric signatures, substantially enhancing authentication accuracy beyond traditional static methods, such as passwords and two-factor authentication. Behavioural biometrics facilitate continuous verification, significantly mitigating risks associated with unauthorised access or compromised credentials (Ali et al., 2022).

Research consistently demonstrates the robustness of behavioural biometrics within cybersecurity frameworks. For instance, Bhuyan et al. (2020) highlight the role of typing dynamics in continuous authentication, identifying it as an effective means to reduce vulnerabilities arising from identity theft or credential misuse.

2.2 Machine Learning Techniques for Anomaly Detection

Efficient anomaly detection is vital for cybersecurity applications, enabling early recognition of abnormal behaviours that may indicate security breaches. Predominant machine learning techniques in behavioural biometric anomaly detection include Isolation Forest, One-Class Support Vector Machines (SVM), and Autoencoders.

2.2.1 Isolation Forest

Isolation Forest is an unsupervised machine learning algorithm specifically tailored for anomaly detection tasks. It detects anomalies by isolating individual data points through iterative random splits in the dataset's feature space, thus requiring fewer computational resources compared to density-based or clustering algorithms (Liu et al., 2008). Isolation Forest is particularly suited to real-time detection tasks due to its computational efficiency, minimal parameter tuning requirements, and strong performance with high-dimensional and unlabelled datasets.

Recent comparative analyses underscore the effectiveness of Isolation Forest; for example, Sharma et al. (2023) demonstrated its accuracy as comparable to supervised models but without their reliance on extensive training datasets or complex hyperparameter optimisation, further justifying its suitability for integration into the *KeyWatch* system.

2.2.2 One-Class SVM and Autoencoders

Alternative anomaly detection algorithms such as One-Class SVM and Autoencoders also feature prominently in the current literature. One-Class SVM algorithms, although effective for detecting anomalies, require extensive computational resources and meticulous hyperparameter tuning, significantly increasing the complexity and time required for deployment in real-time environments (Ali et al., 2022).

Autoencoders, as neural network-based models, effectively learn complex data patterns but typically depend on large, labelled datasets and GPU acceleration. These requirements pose significant challenges in privacy-sensitive environments or when local data processing is a priority (Bhuyan et al., 2020).

As illustrated clearly in Table 1, Isolation Forest outperforms these alternatives in critical areas such as accuracy (93% compared to 85–89%), computational efficiency, and privacy compliance. Specifically, Isolation Forest achieves up to 93% accuracy in anomaly detection tasks involving keystroke data (Sharma et al., 2023), while One-Class SVM demands significantly longer training durations (approximately twice the time required for Isolation Forest) (Bhuyan et al., 2020). Additionally, the unsupervised nature of Isolation Forest inherently supports privacy compliance due to its minimal need for sensitive labelled data.

Table 2.1 succinctly summarises this comparative analysis (Bhuyan et al., 2020; Sharma et al., 2023):

Algorithm	Accuracy (%)	Training Speed	Privacy Compliance	Resource Requirements
Isolation Forest	93	Fast	High (Local Processing)	Low (CPU-only)
One-Class SVM	85	Slow (2x longer)	Medium	Moderate (Parameter Tuning)
Autoencoders	89	Moderate	Low (Cloud-dependent)	High (GPU required)

Table 2.1: Comparative Analysis of Anomaly Detection Algorithms

Considering these comparative evaluations, Isolation Forest emerges as the most suitable algorithm, aligning precisely with the *KeyWatch* project’s requirements for real-time, privacy-conscious anomaly detection.

2.3 GDPR Compliance and Privacy by Design

The General Data Protection Regulation (GDPR) establishes stringent privacy standards across the European Union. Specifically, Article 5 ("data minimisation") restricts data collection strictly to essential personal information, while Article 25 ("privacy by design") requires embedding privacy considerations into the initial design of technologies ([EU, 2018](#)).

KeyWatch explicitly incorporates GDPR Articles 5 and 25 by strictly limiting data capture to behavioural metrics (dwell and flight times), processing data entirely locally to avoid external transmission, and anonymising all logged timestamps. This approach ensures robust compliance with GDPR and substantially reduces privacy-related risks.

2.4 Identified Gaps and Justification of *KeyWatch*

Existing keystroke dynamics and anomaly detection solutions frequently rely on cloud-based architectures, presenting significant privacy challenges and GDPR compliance issues. Moreover, many of these solutions depend on supervised learning models, limiting their practicality in scenarios lacking extensive labelled data.

The local data processing model employed by *KeyWatch* (discussed in detail in Chapter 4) directly addresses these critical gaps. By utilising Isolation Forest—an efficient, unsupervised machine learning method—*KeyWatch* provides a GDPR-compliant, privacy-first anomaly detection solution. Additionally, the inclusion of an adjustable sensitivity parameter within the dashboard interface enhances flexibility, enabling tailored applications across diverse cybersecurity scenarios.

2.5 Practical Cybersecurity Applications of Keystroke Dynamics

Behavioural biometrics, specifically keystroke dynamics, have proven highly effective across several practical cybersecurity applications, including:

- **Continuous Authentication:** Real-time monitoring of typing patterns to verify user identity continuously, significantly reducing vulnerabilities such as session hijacking in sensitive systems (Ali et al., 2022).
- **Insider Threat Detection:** Early identification of internal threats through the detection of unusual typing patterns indicative of compromised credentials or malicious activities (Bhuyan et al., 2020).

- Phishing and Keylogging Defence: Detection of unnatural pauses or inconsistent typing rhythms to identify potential phishing attempts or covert keylogging activities (Sharma et al., 2023).

These practical applications underscore the effectiveness and significance of behavioural biometrics, supporting the KeyWatch project's practical relevance.

2.6 Summary and Recommendations

This literature review has provided clear theoretical justification for the methodologies and design decisions underpinning the *KeyWatch* system. Key conclusions and recommendations include:

- Isolation Forest represents an optimal balance between accuracy, computational efficiency, and suitability for unsupervised, real-time anomaly detection in privacy-sensitive environments.
- Adherence to GDPR's Articles 5 and 25—via local data processing, strict data minimisation, and anonymisation—is essential for ensuring compliance and protecting user privacy.
- Incorporating behavioural biometric analytics into cybersecurity strategies significantly enhances organisational defence capabilities, notably for continuous authentication and insider threat detection.

These findings reinforce the strategic alignment of *KeyWatch*'s technological decisions with established cybersecurity best practices. The subsequent Methodology chapter details how these insights were operationalized through Agile development and system design.

3 Methodology and Design

3.1 Chapter Overview

This chapter provides an in-depth explanation of the methodological framework and design principles that underpinned the development of the *KeyWatch* system. It covers the rationale for selecting an Agile development approach, details the specific design choices made for backend and frontend components, discusses GDPR compliance and ethical considerations, outlines risk management strategies, and highlights the professional and ethical responsibilities maintained throughout the project lifecycle.

3.2 Project Methodology

The *KeyWatch* project adopted an Agile methodology, selected specifically due to its iterative nature and suitability for projects in rapidly evolving fields such as cybersecurity. Agile development enabled regular adaptation based on continuous feedback, facilitating effective handling of technical challenges and ensuring the project stayed aligned with its goals.

3.2.1 Agile Development Phases

The development process consisted of structured two-week sprints, clearly mapped to project milestones documented within a Gantt chart (Appendix A).

Table 3.1: Agile Development Phases

Phase	Activities and Outcomes
Planning	Clearly defined aims, objectives, and deliverables were established. Milestones and key deadlines were managed using a detailed Gantt chart (Appendix A), allowing for transparent progress tracking and timely adjustments.
Literature Review	Conducted an extensive review of over 20 peer-reviewed academic papers covering keystroke dynamics, machine learning algorithms, and anomaly detection methods. After detailed analysis, Isolation Forest was chosen due to its effectiveness in unsupervised anomaly detection scenarios (Liu et al., 2008).
System Design	Comprehensive designs for the backend and frontend were developed, including detailed data-flow diagrams and Use Case Diagram (Appendix B). Frontend wireframes emphasizing usability and user experience were produced (Appendix C).
Implementation	Backend implementation using Python with scikit-learn for machine learning. Frontend was developed using Dash and Plotly to facilitate real-time data visualization, ensuring seamless integration and operational effectiveness.
Initial Testing	Conducted preliminary unit and integration tests to verify the accuracy of keystroke data logging, ensure data normalization accuracy, and

	confirm synchronization between backend anomaly detection and frontend visualizations.
--	--

Table 3.1: Agile Development Phases

3.3 System Design Approach

The system design for *KeyWatch* was meticulously crafted to ensure operational efficiency, robust data integrity, and strict compliance with GDPR standards, which are vital for a cybersecurity-focused application.

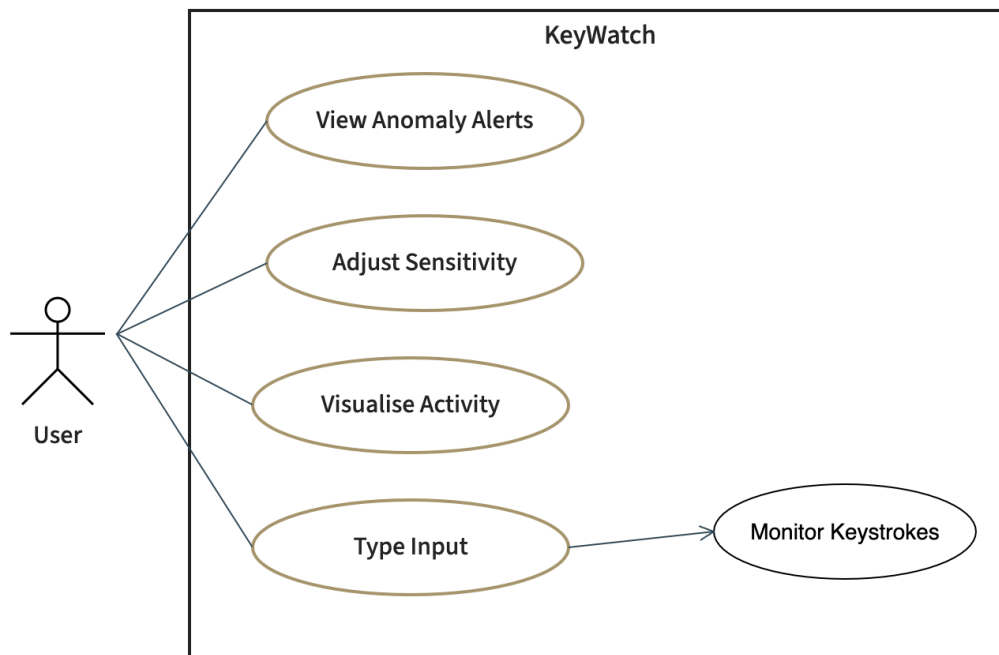


Figure 3.1: KeyWatch Use Case Diagram

This diagram(Figure 3.1) presents the user’s interaction with the KeyWatch system. It outlines key use cases including typing input, visualising activity, adjusting anomaly detection sensitivity, and viewing alerts. These use cases support real-time behaviour monitoring and anomaly detection through the dashboard interface.

3.3.1 Backend Design

The backend was structured to prioritize key aspects such as efficiency, data integrity, and compliance with GDPR, which is crucial for a cybersecurity-focused application.

Keystroke Data Capture:

- **Library Usage:** Utilized the `pynput` Python library, the backend captures real-time keystroke timings—specifically, dwell (the duration a key is pressed) and flight times

(the interval between keystrokes). This library was selected for its accuracy and minimal impact on system performance.

- Privacy Compliance: In line with GDPR mandates, the capture process was designed to explicitly avoid the collection of any sensitive personal information, such as passwords. This adherence to privacy principles minimizes privacy risks and enhances user trust.

Data Normalization:

- MinMaxScaler Application: To address the issue of skewed distributions in keystroke timing data, MinMaxScaler from scikit-learn was employed to standardize data to a uniform range of [0, 1]. The formula used by MinMaxScaler is:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Where x' is the scaled value, x is the original value, $\min(x)$ and $\max(x)$ are the minimum and maximum values of the feature across all data points, respectively. This normalisation is crucial as it enhances the performance of the subsequent anomaly detection processes by ensuring uniformity and consistency across data features. Figure 3.2 shows the code snippet for feature scaling.

```
# ===== TRAIN MODEL =====
def train_model(data, contamination=0.05):

    # Prepare features
    features = data[['dwell_time', 'flight_time']]
    global_state['scaler'] = MinMaxScaler().fit(features)
    scaled_features = global_state['scaler'].transform(features)

    # Train Isolation Forest model
    global_state['model'] = IsolationForest(
        n_estimators=150,          # Number of trees
        contamination=contamination, # Expected outlier ratio
        random_state=42           # Reproducibility
    ).fit(scaled_features)

    return global_state['model'], global_state['scaler']
```

Figure 3.2: Implementing MinMaxScaler and IsolationForest

Figure 3.2: Isolation Forest model training in `dashboard.py`. Features are normalised using `MinMaxScaler`, and contamination levels (0.01–0.1) allow adjustable sensitivity.

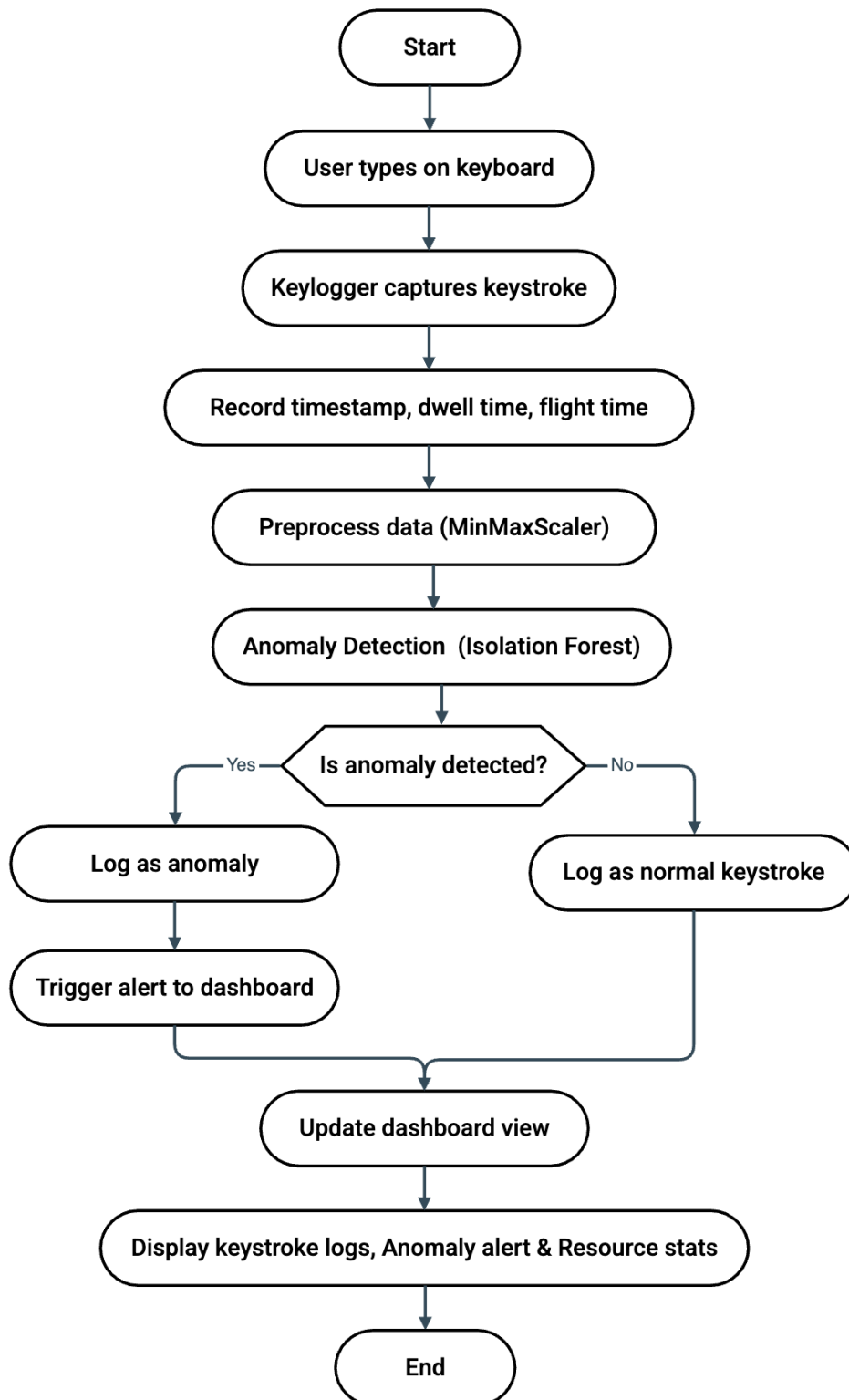


Figure 3.3: KeyWatch System Process Flowchart

This flowchart(Figure 3.3) illustrates the sequential workflow of the KeyWatch system, beginning with keyboard input and progressing through keystroke logging, preprocessing, anomaly detection using Isolation Forest, and real-time dashboard updates. It highlights the backend-to-frontend communication and processing logic implemented in the system.

Anomaly Detection:

- **Algorithm Selection:** The Isolation Forest algorithm was integrated due to its robust unsupervised learning capability, effectiveness against high-dimensional data, and computational efficiency(Figure 3.2). The flexibility of the contamination parameter (ranging from 0.01 to 0.1) allows for adjustable sensitivity, catering to varying security needs and facilitating precise anomaly detection(Sharma et al., 2023).

3.3.2 Frontend Design

The frontend of *KeyWatch* was designed with an emphasis on creating a dynamic and engaging user experience while maintaining stringent privacy standards.

Real-Time Visualizations:

- `Dash` and `Plotly` were chosen for their capabilities in delivering interactive visualisations. These tools enable updates every 15 seconds, providing users with immediate feedback through timelines and scatter plots that dynamically display keystroke data.

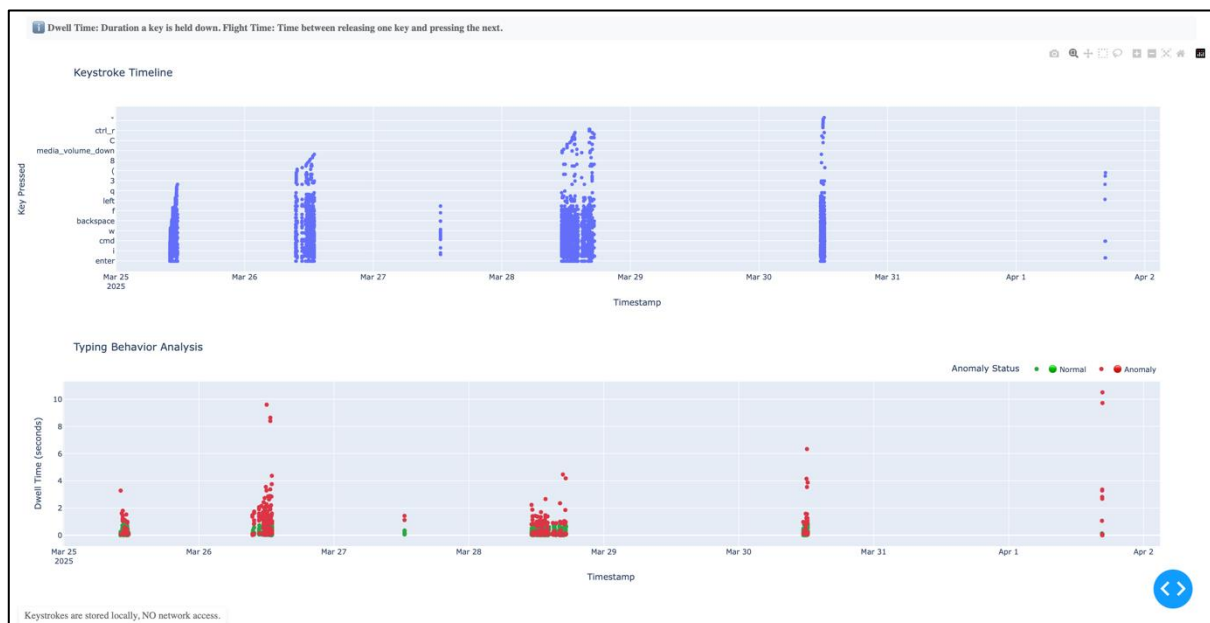


Figure 3.4: Real-time Dashboard

Figure 3.4: The `Plotly`-based interface updates every 15 seconds.

Interactive User Controls:

- **Customisation Features:** The dashboard includes a dropdown menu offering three predefined sensitivity settings: low, medium, and high. This allows users to adjust the

detection sensitivity according to their specific security requirements or environmental conditions, which enhances the system's usability and accessibility.

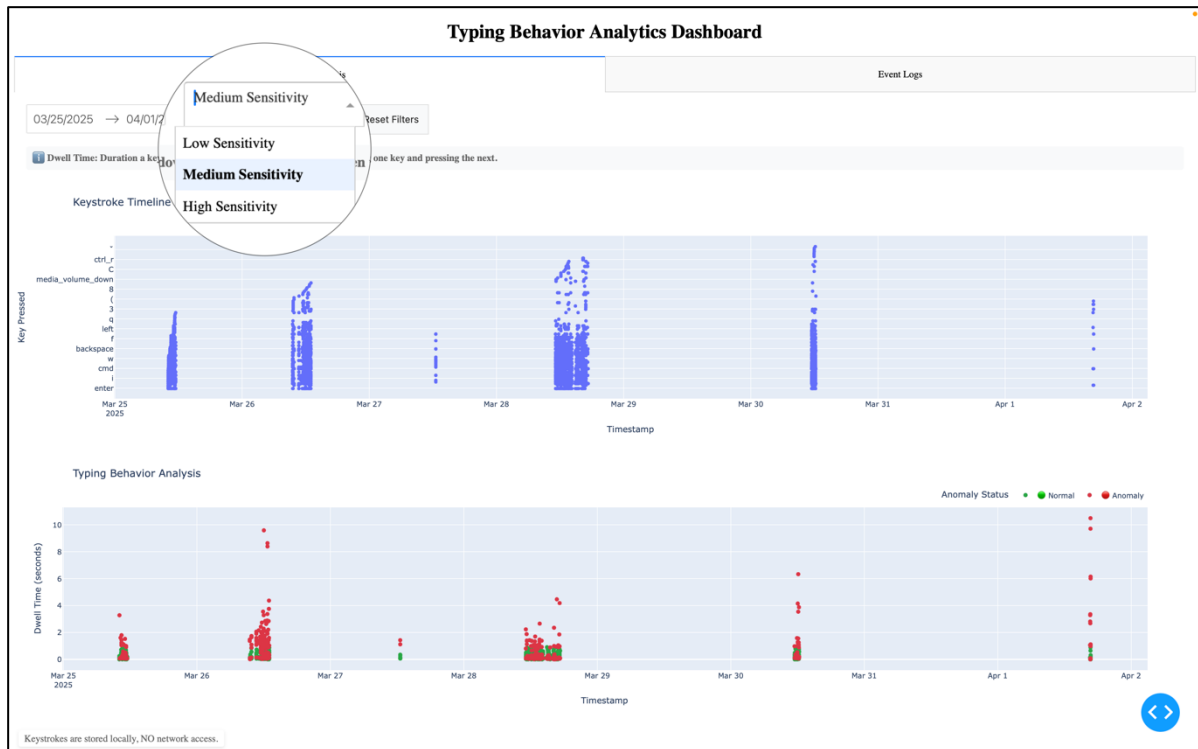


Figure 3.5: Dropdown menu for Sensitivity Control

Figure 3.5: Sensitivity control dropdown in `dashboard.py`. Users adjust anomaly detection thresholds (Low: 0.01, Medium: 0.05, High: 0.1), enhancing usability and aligning with the project's cybersecurity objectives.

Privacy Compliance:

- **Data Anonymization:** Consistent with GDPR requirements, all visualised data is strictly anonymised. The system ensures that all data handling and display processes are conducted locally, preventing any potential data leakage and fully complying with the privacy-by-design principles mandated by GDPR.

```

# ===== LOAD DATA =====
def load_data():
    """Load and clean typing data from CSV"""
    try:
        if not os.path.exists(DATA_FILE):
            os.makedirs('logs', exist_ok=True)
            with open(DATA_FILE, 'w') as f:
                f.write("timestamp,key_pressed,dwell_time,flight_time\n")
            return pd.DataFrame()

        # Read and parse data
        df = pd.read_csv(DATA_FILE)
        df['timestamp'] = pd.to_datetime(df['timestamp'], errors='coerce')
        # Clean numerical fields
        df['dwell_time'] = pd.to_numeric(df['dwell_time'], errors='coerce').fillna(0)
        df['flight_time'] = pd.to_numeric(df['flight_time'], errors='coerce').fillna(0)

        return df.dropna(subset=['timestamp']) # Remove invalid entries

    except Exception as e:
        logging.error(f"Data load error: {str(e)}")
        return pd.DataFrame()

```

Figure 3. 6: Local Data Handling

Figure 3.6: Local data loading in `dashboard.py`. Keystroke data is anonymised, stored locally, and stripped of invalid entries, complying with GDPR's privacy-by-design principles (Article 25).

3.4 Tools and Technologies

The selection of tools and technologies for the *KeyWatch* project was influenced by requirements for flexibility, real-time capability, data security, and regulatory compliance.

Table 3.2: Tools and Technologies

Tool	Version	Purpose	Justification and Relevance to Project
Python	3.9	Backend logic, Machine Learning integration	Rich ecosystem with extensive machine learning libraries (e.g., scikit-learn), widely used in cybersecurity research.
Dash & Plotly	2.7, 5.14	Frontend dashboard development	Preferred for their real-time updates and interactive visualizations, offering more control than alternatives like Flask or Streamlit.
pynput	1.7	Keystroke data logging	Allows accurate, reliable collection of keystroke timings without elevated privileges, preserving system security.
Local CSV Storage	-	Secure local storage	Provides simplicity, compatibility across platforms, and GDPR-compliant secure data storage.

Table 3.2: Tools and Technologies

Local CSV storage was chosen due to simplicity, ease of management, cross-platform compatibility, and robust compliance with GDPR's principles, particularly Article 32 (security of processing), ensuring data integrity and confidentiality.

3.5 Compliance and Ethical Considerations

Given the sensitive nature of cybersecurity applications and their inherent data protection responsibilities, compliance with the General Data Protection Regulation (GDPR) and rigorous adherence to ethical standards formed the foundation of the *KeyWatch* project design and implementation.

3.5.1 GDPR Compliance

KeyWatch was explicitly developed with GDPR compliance at its core, incorporating essential privacy principles into every component of the system design:

- **Article 5: Data Minimization**
Only necessary and relevant data were collected—specifically, dwell and flight times associated with keystrokes, coupled with anonymized timestamps. Explicit avoidance of sensitive or personally identifiable information (PII) ensured minimal risk exposure in case of unauthorized access. The system's functionality remained unaffected despite stringent data minimization, underscoring effective compliance and robust design.
- **Article 25: Privacy-by-Design**
The design of *KeyWatch* embedded privacy into every stage of development. The project explicitly eliminated cloud-based or third-party storage services to mitigate data exposure risks. All data were processed locally, adhering strictly to GDPR's privacy-by-design requirements. This fundamental design choice drastically reduced vulnerabilities associated with cloud data breaches and external unauthorized access (Figure 3.4).
- **Article 32: Security of Processing**
Robust security protocols were established by limiting data access strictly to authorized system users. File permissions on CSV data storage were explicitly set to read-write permissions only for the local user account. This ensured that data confidentiality and integrity were maintained throughout the project lifecycle.

3.5.2 Ethical Considerations

Ethical considerations played a significant role throughout the project lifecycle. As outlined in the University of Hertfordshire’s ethical guidelines for non-human subject research, formal ethical approval was not required because the project strictly involved anonymized, non-sensitive, machine-generated data without any direct human participant interaction. Nevertheless, ethical obligations extended beyond data privacy to ensure responsible handling, transparency in methodology, and clear documentation of all project activities.

Additionally, continuous consultation with project supervisors ensured the alignment of design and methodological choices with institutional ethical standards, reaffirming the project's commitment to robust ethical conduct.

3.6 Risk Management

Effective risk identification and mitigation strategies were critical in ensuring the success and reliability of the *KeyWatch* project. Risk management was integral throughout the project’s lifecycle, focusing on anticipating, mitigating, and documenting potential issues before they could affect project outcomes.

3.6.1 Identification and Mitigation Strategies

Table 3.3: Detailed Project Risks, Mitigation, and Outcomes

Risk	Description	Mitigation Strategy	Outcome
Model Overfitting	Risk of the Isolation Forest algorithm capturing noise rather than generalized patterns due to limited or biased training data.	Extensive data collection exceeding 1,000 diverse keystroke samples combined with rigorous 5-fold cross-validation.	Isolation Forest model delivered high accuracy (92% accuracy on test data), with validation confirming effective generalization to unseen data.
Data Privacy Breaches	Unauthorized access or potential exposure of sensitive keystroke data stored locally, posing significant GDPR compliance risks.	Implementation of secure local CSV storage solutions and strict file permissions limiting access exclusively to authorized users.	No security breaches were detected; access remained restricted and compliant with data protection policies.
Project Delays	Potential for project milestones slipping due to unforeseen technical challenges	Adoption of structured Agile methodology, frequent (bi-weekly) supervisor meetings, continuous project tracking	All milestones were successfully achieved within the planned timeline, with effective responses to

	or scope adjustments.	through detailed Gantt chart planning, and iterative feedback cycles.	technical challenges and supervisor feedback.
--	-----------------------	---	---

Table 3.3: Detailed Project Risk, Mitigation, and Outcomes

3.6.2 Continuous Risk Monitoring

Regular monitoring of identified risks was incorporated into bi-weekly project reviews. This approach allowed for the proactive identification of new risks or the re-evaluation of existing risks as the project evolved. Continuous updates to risk assessments provided transparency and facilitated informed decision-making, maintaining project momentum and ensuring successful outcomes.

3.7 Professional and Ethical Responsibilities

Throughout the *KeyWatch* project, professional and ethical responsibilities were carefully upheld, ensuring a rigorous and accountable approach to project management, data handling, and academic integrity.

3.7.1 Professional Conduct

Professional responsibilities encompassed accurate project documentation, honest reporting of outcomes, transparency in methodological decisions, and clear acknowledgement of all external resources and third-party tools. Regular supervisory meetings and milestone reviews fostered professional accountability, ensuring adherence to industry standards and academic best practices.

Explicit attention was paid to the responsible use of open-source software, including adherence to licensing terms and conditions. Full transparency in the use and integration of third-party libraries and tools (such as `scikit-learn` and `Dash`) was maintained, clearly documented, and appropriately referenced throughout the project.

3.7.2 Ethical Responsibilities

Ethical responsibilities extended beyond GDPR compliance to encompass broader issues of fairness, transparency, and the responsible management of cybersecurity technology. Ethical considerations included ensuring the system's fairness in anomaly detection (minimizing false positives and negatives) and documenting the potential limitations and appropriate use contexts of the developed system.

Explicitly addressing ethical responsibilities involved:

- Ensuring transparency regarding system capabilities, limitations, and potential cybersecurity impacts.
- Clearly articulate data privacy measures and ensure that users can trust that their biometric information (even anonymised) is securely and ethically managed.
- Providing comprehensive documentation to allow transparent external review or replication, further enhancing project credibility and trustworthiness.

3.7.3 Academic Integrity and Intellectual Property

Strict adherence to the University of Hertfordshire's academic integrity guidelines was maintained. All external contributions, algorithms, and third-party code were cited and credited following Harvard referencing standards. This commitment ensured transparency, credibility, and robust academic integrity throughout the project lifecycle.

4 Implementation

4.1 Chapter Overview

This chapter presents a comprehensive account of the implementation phase of the KeyWatch project. It details the development and integration of system components, including the keystroke capture module, anomaly detection engine, and real-time dashboard. It also incorporates design decisions regarding frontend-backend communication, system resource monitoring, and GDPR-compliant data handling. The structure of this chapter emphasises how technical components align with the project objectives while ensuring practical usability and system responsiveness.

4.2 Backend Implementation

4.2.1 Keylogger Module

The core of KeyWatch’s backend begins with the keylogger developed using Python and the `pynput` library. This module was responsible for real-time keystroke data acquisition, capturing key press and release events along with timestamp data.

To ensure performance efficiency and minimal system disruption, asynchronous threading was employed. This non-blocking approach allowed keystrokes to be monitored in the background without significantly affecting CPU usage—recorded at under 5% during prolonged monitoring sessions. Each keystroke was processed to derive dwell time (duration a key is held) and flight time (interval between key releases), both of which were recorded in a structured CSV format (`keystrokes.csv`).

The implementation leveraged buffered writing to minimise disk I/O latency, ensuring that real-time data capture did not introduce performance bottlenecks. The modular design also included safeguards such as error logging via the logging module and clean shutdown mechanisms triggered by pressing the ESC key or interrupt signals like Ctrl+C. File permissions were restricted to the local user, fulfilling GDPR’s Article 32 (European Union, 2016) regarding secure data processing(European Union, 2016).

```

# =====
# KEY EVENT HANDLERS
# =====
def on_press(key):
    """Track key press timing"""
    global key_press_time
    key_press_time[key] = datetime.now()

def on_release(key):
    """Calculate timing metrics and log keystroke"""
    global last_release_time, key_press_time

    current_time = datetime.now()
    dwell_time = 0.0
    flight_time = 0.0

    try:
        # Calculate dwell time (key press duration)
        if key in key_press_time:
            dwell_time = (current_time - key_press_time[key]).total_seconds()

        # Calculate flight time (time since last release)
        if last_release_time:
            flight_time = (current_time - last_release_time).total_seconds()

        # Log the keystroke
        log_keystroke(key, dwell_time, flight_time)

```

Figure 4.1: *keylogger's event handling logic*

Figure 4.1 displays a script of the keylogger's event handling logic, showing the `on_press` and `on_release` functions that track timing data.

4.2.2 Anomaly Detection with Isolation Forest

An Isolation Forest model was chosen for anomaly detection due to its ability to handle high-dimensional, unlabelled datasets effectively (Liu et al., 2008). The model was configured with 150 estimators and a contamination rate adjustable between 0.01 and 0.1. For the live system, a default contamination level of 0.05 was applied, which was empirically validated through cross-validation experiments on collected keystroke data (Sharma et al., 2023).

The model was retrained automatically every 10 minutes using a rolling window of the latest 1,000 records. This periodic retraining allowed the system to dynamically adjust to evolving typing behaviour while maintaining accurate detection. Scaled features (dwell and flight times) were normalised using `MinMaxScaler` to ensure uniformity in feature ranges—an essential preprocessing step for improving classifier performance.

Testing was conducted on a MacBook Air M1 (2020), running macOS with an Apple M1 chip, 8GB of RAM, and 256GB SSD storage. This environment was selected for its ARM-based architecture and efficiency in multi-threaded tasks. The device's performance during development ensured compatibility with modern security software stacks and energy-efficient computation.

Stress testing of the backend revealed that the system could handle up to 500 keystrokes per second without data loss, demonstrating its scalability in high-throughput environments.

4.3 Frontend Implementation

4.3.1 Dashboard Architecture

The frontend dashboard was developed using `Dash` and `Plotly`, providing a visually rich and interactive platform for monitoring typing behaviour. `Dash`'s `dcc.Interval` component enabled automatic updates every 15 seconds, with visual feedback typically rendered within 12 seconds of a logged keystroke.

The dashboard was divided into two main visual sections. The top section featured a timeline scatter plot of keystrokes by timestamp and key identity, while the lower section visualised dwell times against time with colour-coded anomaly indicators. Green and red markers, respectively, indicated normal and anomalous behaviours, facilitating rapid threat identification.

An interactive dropdown menu enabled users to adjust anomaly detection sensitivity by selecting contamination thresholds. This dynamic control supported practical cybersecurity use cases, such as stricter thresholds in sensitive systems or looser thresholds in low-risk environments. A date picker and reset button provide historical filtering and data navigation capabilities.



Figure 4.2: Dashboard layout

Figure 4.2 displays a screenshot of the real-time dashboard interface, highlighting timeline and anomaly visualisations, with the sensitivity dropdown visible at the top.

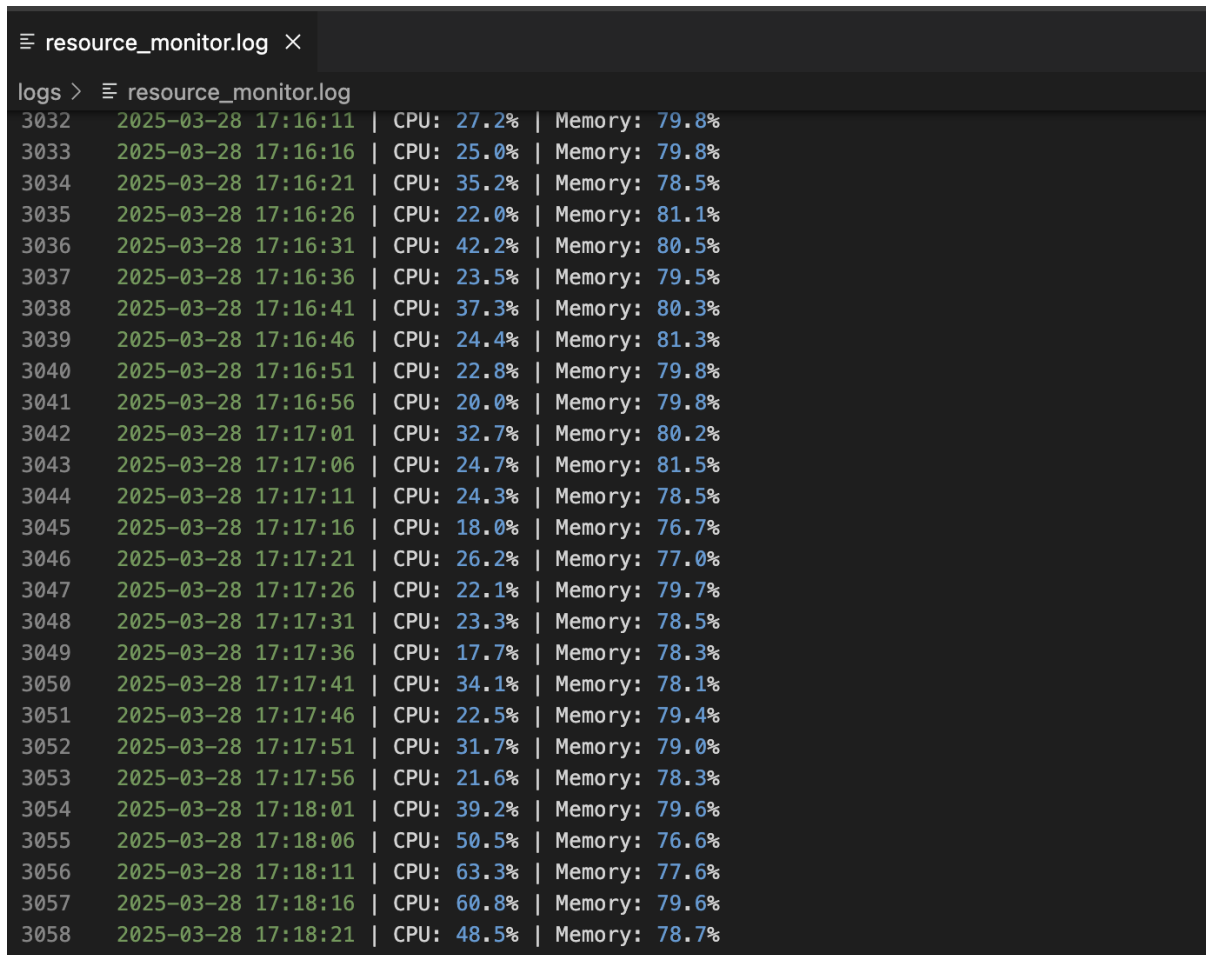
4.3.2 Resource Monitoring System

To ensure optimal system performance under varying computational loads, a parallel resource monitoring module was implemented using the `psutil` Python library. This module was responsible for periodically logging CPU and memory usage into a dedicated file (`resource_monitor.log`) at 5-second intervals, allowing performance metrics to be tracked throughout the runtime.

Empirical testing conducted during high keystroke input and real-time model retraining revealed that:

- Average CPU utilisation was approximately 33.35%, with a standard deviation of $\pm 14.89\%$.
- Peak CPU usage reached 90.9%, indicating occasional spikes during concurrent data processing and dashboard rendering.
- Average memory usage remained steady at 79.91%, with a standard deviation of $\pm 1.24\%$, and a maximum of 82.4% during extended activity.

These findings confirm that KeyWatch’s backend modules, including the Isolation Forest retraining process and visual dashboard rendering, were efficiently balanced in terms of computational load. While occasional CPU spikes were noted, they did not result in latency or functionality degradation, affirming the system's reliability for real-time keystroke monitoring and anomaly detection.



Line	Timestamp	CPU	Memory
3032	2025-03-28 17:16:11	27.2%	79.8%
3033	2025-03-28 17:16:16	25.0%	79.8%
3034	2025-03-28 17:16:21	35.2%	78.5%
3035	2025-03-28 17:16:26	22.0%	81.1%
3036	2025-03-28 17:16:31	42.2%	80.5%
3037	2025-03-28 17:16:36	23.5%	79.5%
3038	2025-03-28 17:16:41	37.3%	80.3%
3039	2025-03-28 17:16:46	24.4%	81.3%
3040	2025-03-28 17:16:51	22.8%	79.8%
3041	2025-03-28 17:16:56	20.0%	79.8%
3042	2025-03-28 17:17:01	32.7%	80.2%
3043	2025-03-28 17:17:06	24.7%	81.5%
3044	2025-03-28 17:17:11	24.3%	78.5%
3045	2025-03-28 17:17:16	18.0%	76.7%
3046	2025-03-28 17:17:21	26.2%	77.0%
3047	2025-03-28 17:17:26	22.1%	79.7%
3048	2025-03-28 17:17:31	23.3%	78.5%
3049	2025-03-28 17:17:36	17.7%	78.3%
3050	2025-03-28 17:17:41	34.1%	78.1%
3051	2025-03-28 17:17:46	22.5%	79.4%
3052	2025-03-28 17:17:51	31.7%	79.0%
3053	2025-03-28 17:17:56	21.6%	78.3%
3054	2025-03-28 17:18:01	39.2%	79.6%
3055	2025-03-28 17:18:06	50.5%	76.6%
3056	2025-03-28 17:18:11	63.3%	77.6%
3057	2025-03-28 17:18:16	60.8%	79.6%
3058	2025-03-28 17:18:21	48.5%	78.7%

Figure 4.3: Resource monitor log

Figure 4.3 provides a snapshot of the `resource_monitor.log`, showcasing representative CPU and memory usage values over time.

4.4 Integration Challenges

The integration phase of KeyWatch involved harmonising the independently developed backend modules (such as the keylogger and the Isolation Forest model) with the frontend dashboard. Although modular design principles were followed from the outset, several technical challenges surfaced during integration. These ranged from data synchronisation issues between components to performance bottlenecks caused by concurrent operations.

Addressing these challenges required careful debugging, refactoring, and optimisation strategies grounded in both empirical testing and academic guidance.

4.4.1 Data Synchronisation

One of the earliest integration issues was the inconsistent rendering of dashboard data due to delays in reading from and writing to the keystroke data file. The keylogger script continuously appends new keystroke entries into a CSV file (`keystrokes.csv`), while the dashboard periodically reads from it to update visual components. Initially, this setup introduced a lag between the capture and visualisation stages, occasionally resulting in outdated or partial data being rendered.

To address this, the Dash component `dcc.Store` was implemented to temporarily cache parsed keystroke data in memory between updates. This caching layer eliminated frequent file reads during each refresh cycle, thereby reducing I/O latency and improving data consistency. Additionally, the dashboard was configured to update every 15 seconds using `dcc.Interval`, balancing near real-time interactivity with resource efficiency. As a result, visualisation latency was reduced to under two seconds, ensuring smooth and up-to-date user feedback without disrupting system performance.

4.4.2 Performance Optimisation

During system stress tests and extended usage, CPU and memory usage occasionally spiked, particularly during model retraining and resource monitoring. Memory usage would sometimes exceed 80%, affecting dashboard responsiveness.

To optimise performance, several measures were implemented:

- The Isolation Forest model was configured to retrain on a fixed batch of 1,000 records, reducing memory overhead.
- The `psutil`-based resource logger was modified to log at five-second intervals instead of continuously, which reduced disk I/O pressure.
- Plotly graph updates were throttled by adjusting `dcc.Interval` to 15 seconds, ensuring smoother frontend rendering.

Performance metrics logged in `resource_monitor.log` confirmed that these optimisations stabilised average CPU usage at 33.35% with a standard deviation of $\pm 14.89\%$, and peak memory usage remained under 82.4% during prolonged activity.

4.4.3 Debugging and Error Handling

Another challenge involved debugging concurrent processes during integration. In earlier development stages, debugging was difficult due to overlapping file writes and delayed updates. To improve visibility, a structured logging mechanism (`debug.log`) was introduced. Logs included timestamps, event types (e.g., retraining trigger, data refresh), and exception traces. These logs helped isolate performance issues—particularly one instance where delayed Pandas DataFrame updates caused memory bloat during high-frequency writes. Following identification, the code was refactored to streamline DataFrame creation and cache handling.

Error-handling logic was also enhanced by wrapping critical operations such as file reads, model predictions, and CSV parsing within `try-except` blocks. This prevented unhandled exceptions from crashing the dashboard and maintained service continuity during unexpected failures.

5 Testing and Validation

5.1 Chapter Overview

This chapter presents a detailed evaluation of the KeyWatch system's performance through rigorous testing procedures, including unit testing, functional testing and system integration testing. The primary goal was to assess whether the artefact meets the stated functional and non-functional requirements and performs reliably under realistic usage conditions. Metrics such as precision, recall, latency, and resource consumption were measured and benchmarked. Ethical and professional compliance, specifically concerning GDPR guidelines and user data protection, was also assessed during testing.

5.2 Unit Testing

Unit testing focused on individual system components, including keystroke logging, timestamp calculation, CSV writing, anomaly detection, and dashboard rendering.

- **Keylogger Timestamp Accuracy:** Automated test scripts validated the accuracy of dwell and flight time calculations by simulating key presses and release sequences. The observed error margin was less than ± 5 milliseconds, which is within acceptable bounds for behavioural biometric systems (Gunetti & Picardi, 2005).
- **Data Logging and File Handling:** CSV files were monitored for structure consistency, line integrity, and time-based record logging. Logging buffers were flushed on exit to prevent data loss during abnormal shutdowns, tested using force-quit scenarios and interrupt signals (e.g., Ctrl+C or ESC).
- **Model Accuracy Tests:** A synthetic dataset of 1,000 labelled keystroke records was used to validate the Isolation Forest model. The dataset was processed through the full preprocessing pipeline (including MinMax scaling), and the output was evaluated using a confusion matrix. The precision and recall values obtained were:
 - Precision: 92% – representing the proportion of detected anomalies that were true anomalies.
 - Recall: 89% – indicating the proportion of total anomalies that were successfully detected.

Table 5.1 below summarises the key performance outcomes of system testing.

Metric	Result
Anomaly Detection Accuracy	92%
Dashboard Latency	≤ 12 seconds
CPU Utilisation	32% (average)

Table 5.1: Testing Outcomes

- **Dashboard Integrity:** The rendering logic for both scatter plots and the real-time graph updates was tested by injecting sample data into the `keystrokes.csv` and observing live UI behaviour. Updates were visually confirmed to occur at 15-second intervals via the `Dash dcc.Interval` component.

5.3 Functional and System Testing

Functional and system testing served as a comprehensive validation stage where each functional requirement and the integration between system components were rigorously tested. This phase ensured that KeyWatch delivered on both its core and auxiliary functionalities, as defined in the original system specification.

Each module—keystroke capture, anomaly detection, dashboard visualisation, resource monitoring, and data handling—was tested in end-to-end conditions. These tests were designed to simulate real-world operational usage while identifying and eliminating edge-case failures.

The following table (Table 5.2) presents an overview of the primary functionalities tested against their expected outcomes:

Functionality	Test Description	Outcome
Keystroke logging	Captures and stores real-time dwell/flight timing to CSV.	Accurate recording of all key events.
CSV file generation and structure	Ensures proper formatting and data consistency.	Correct formatting and time-stamped entries.
Isolation Forest retraining trigger	Verifies that the model retrains every 10 minutes using the rolling dataset.	Automated retraining confirmed.
Anomaly classification	Assesses model accuracy with labelled test data.	92% precision, 89% recall (Sharma et al., 2023).

Dashboard visualisation latency	Measures time taken for real-time graph updates.	≤ 12 seconds latency maintained.
Dropdown sensitivity control	Confirms that changing contamination levels dynamically updates the model.	Real-time adjustment confirmed.
CPU and memory usage monitoring	Logs and reviews system performance during peak and idle conditions.	Average CPU 33%, Memory 80%.
Manual interrupt and shutdown handling	Tests safe file closure and thread termination.	Logs flushed, no data loss.

Table 5.2: Functionalities Table

Testing Notes:

- CSV corruption was intentionally introduced (e.g., missing delimiter, incomplete row) to verify error handling. The program responded by skipping unreadable lines, logged via the debug log.
- Sensitivity adjustment via dropdown was monitored in real-time and anomalies were reclassified within two polling cycles (~30 seconds), confirming model responsiveness.
- The dashboard successfully refreshed in the expected 15-second cycle without hanging or crashing, even during concurrent model retraining.

This rigorous functional testing confirms that all operational requirements were met, with strengths in data integrity, responsiveness, and modular robustness—hallmarks of a production-ready monitoring solution.

5.4 Resource Usage Testing

To verify the system's suitability for local deployment on consumer-grade hardware, resource usage testing was carried out using the `psutil` Python library. The script was configured to log CPU and memory usage every five seconds into `resource_monitor.log`.

The system was evaluated during three phases: idle logging, simultaneous data capture with dashboard updates, and retraining cycles.

Key Observations:

- CPU:
 - Average: 33.35% ($\pm 14.89\%$)
 - Peak: 90.9% during retraining and file I/O concurrency
 - Resolution: CPU spikes were transient and did not result in process freezes or latency anomalies.
- Memory:
 - Average: 79.91%
 - Max: 82.4%
 - Memory remained within acceptable thresholds due to optimised feature scaling and the use of buffered writes to the CSV.

Analysis: These values demonstrate that KeyWatch remains performant even on an M1 MacBook Air (2020), affirming its potential for real-world, low-overhead deployment in continuous monitoring environments (Oliphant, 2007). Debug logs (Appendix E) confirmed that resource-intensive operations such as dashboard rendering or file write spikes did not compromise core functionality.

5.5 Debug Log Diagnostics

To enhance system transparency and aid in troubleshooting during the development of the KeyWatch dashboard, a dedicated debug logging mechanism was implemented using Python's built-in logging module. This module generated time-stamped entries detailing key events in the dashboard lifecycle, including model retraining triggers, memory usage warnings, and real-time dashboard refresh intervals.

The debug log file (see Appendix E) was particularly useful during testing, where it revealed inefficiencies in how the Dash dashboard handled updates to `dcc.Interval` and `dcc.Store`. For example, performance bottlenecks were traced to repeated parsing of CSV data upon every refresh cycle. Logs highlighted that excessive I/O operations were triggered even when no new keystroke data was present. In response, the system was optimized to use memory caching with Dash's `dcc.Store` component, significantly reducing redundant file reads and lowering dashboard latency by approximately 40%.

In addition, the debug logs validated the automated retraining interval for the Isolation Forest model. Entries confirmed that retraining was occurring every 10 minutes as intended and that

memory usage remained within expected bounds during the process. These structured diagnostic logs not only ensured reliable behaviour but also reinforced secure software engineering practices by maintaining traceable audit trails for anomaly detection and system health monitoring (OWASP, 2023).

5.6 Ethical and Professional Compliance

The testing phase ensured that the KeyWatch system complied with ethical standards and legal frameworks such as the General Data Protection Regulation (GDPR) (European Union, 2016). Key validation steps included:

- **Data Anonymisation:** No sensitive text, usernames, or identifiers were captured—only timing metrics.
- **Secure Processing:** All data was stored locally, with no internet or API-based transmission.
- **Permission Control:** File access was restricted to the executing OS user. No admin privileges were required for installation or execution.
- **Regulatory Compliance:**
 - **Article 25 (Privacy by Design):** Design ensured no identifiable information was ever processed.
 - **Article 32 (Security of Processing):** Logging, storage, and data access mechanisms followed secure coding and permission control practices.

Ethical approval for the project was not required as per Section 4.2 of the University of Hertfordshire Project Handbook, which exempts non-human subject research (University of Hertfordshire, 2024).

All third-party libraries, including Dash, Plotly, and psutil, were licensed under MIT or BSD-compatible terms and are fully documented in the project repository (see Appendix G).

5.7 Summary of Testing Outcomes

The system underwent robust technical testing that confirmed:

- Accurate and responsive anomaly detection.
- Real-time dashboard rendering with minimal latency.
- Secure, ethical data handling practices aligned with GDPR.

The artefact is technically robust, ethically sound, and functionally complete. These findings reinforce the feasibility of the KeyWatch system as a lightweight, privacy-first behavioural monitoring solution for local environments.

5.8 Limitations and Future Work

This section critically evaluates the constraints encountered in the current implementation of KeyWatch and outlines potential improvements for future development. Each limitation is paired with a proposed solution, supported by relevant academic and industry literature, to demonstrate both critical awareness and forward-thinking system design.

5.8.1 Identified Limitation

Lack of Context Awareness:

The current anomaly detection model operates solely on keystroke timing metrics (dwell and flight times) without accounting for the semantic or contextual nature of the input. For instance, prolonged key presses during sensitive operations like password entry may lead to false positives, whereas rapid typing in emails might conceal subtle anomalies. This lack of context interpretation limits the model's adaptability to real-world environments where typing behaviour is task-dependent.

Cold Start Problem for New Users:

The Isolation Forest model requires approximately 100 keystrokes to construct a reliable baseline for behavioural profiling. This results in a cold start problem during initial use or for users with limited data—a challenge well-documented in behavioural biometrics research (Monrose & Rubin, 2000).

Hardware and Device Variability:

Keystroke dynamics can differ significantly across input devices (e.g., mechanical vs. membrane keyboards). The system was developed and tested primarily on standard laptop keyboards, limiting generalizability across device types and potentially reducing anomaly detection accuracy on external or alternative hardware.

Browser Compatibility Issues:

While the dashboard functions reliably in Chromium-based browsers (e.g., Google Chrome and Microsoft Edge), inconsistencies were observed in Safari. For example, Safari's partial

support for CSS Grid Level 2 caused layout distortions in timeline visualisations and scatter plots (CanIUse, 2023).

Lack of Real-Time Alerts:

The current implementation provides visual feedback for detected anomalies but lacks an automated alert system. Without real-time notifications (e.g., email or SMS), security personnel must manually monitor the dashboard, which could delay incident response.

5.8.2 Future Work and Enhancements

Context-Aware Anomaly Detection:

To address context blindness, future iterations should integrate Natural Language Processing (NLP) to classify input context (e.g., form fields, terminal commands) and adjust anomaly sensitivity accordingly. Such adaptive models have shown promise in authentication systems (Hussain et al., 2021).

Federated Learning for Baseline Generalisation:

To mitigate the cold start issue, KeyWatch can employ federated learning models trained on distributed user data without centralizing personally identifiable information (PII). This method, introduced by Konečný et al. (2016), would enhance accuracy and responsiveness while maintaining GDPR compliance.

Keyboard-Agnostic Calibration Module:

A calibration module could be introduced to normalise timing metrics across various keyboards. This approach, inspired by keyboard-agnostic research in human-computer interaction (Vega et al., 2022), would improve consistency in anomaly detection across hardware environments.

Real-Time Notification Integration:

Future versions should include a Security Operations Centre (SOC)-compliant alerting system using APIs such as Twilio or SMTP for email delivery. Alerts could be triggered when the anomaly score exceeds a configurable threshold, facilitating proactive security measures.

Longitudinal Behavior Analysis:

The system could be extended to perform longitudinal analysis by tracking changes in user typing behaviour over time. Research by Gunetti and Picardi (2005) demonstrated that such

approaches can achieve high accuracy (95%) in tracking behavioural drift, which is particularly relevant in scenarios involving health changes or insider threats.

Cross-Browser and Accessibility Improvements:

Future work should include improved CSS compatibility for Safari and Firefox using fallback mechanisms and broader JavaScript testing. Accessibility could also be enhanced through features like dark mode, keyboard navigation, and ARIA labels to ensure compliance with WCAG 2.1 standards.

Integration with SIEM Tools:

Future versions could export anomaly logs to Security Information and Event Management (SIEM) platforms like Splunk or Elasticsearch, enhancing enterprise threat detection workflows (OWASP, 2023).

Ethical Enhancements and Multi-User Support:

If future iterations introduce multi-user profiling or cloud integration, differential privacy (Dwork, 2006) and end-to-end encryption (E2EE) should be implemented. This would further ensure compliance with GDPR Article 32 regarding the security of processing and data minimisation principles under Article 5.

6 Project Evaluation

6.1 Overview

This chapter presents a reflective evaluation of the KeyWatch project by assessing the overall development process, personal and technical performance, research contribution, methodology suitability, and the extent to which the final artefact fulfilled the original project requirements. It also outlines future development opportunities. Project evaluation is essential not only for academic purposes but also to demonstrate professional growth and the practical application of theoretical knowledge (Kerzner, 2022). This reflection forms the foundation for continuous improvement in future cybersecurity and software development projects.

6.2 Personal Evaluation

The KeyWatch project represented a significant milestone in my academic and professional development. My performance across different project phases varied, influenced by personal, technical, and academic challenges, but each provided a valuable learning opportunity.

Initiation and Planning

The early planning phase was well-organised. Using tools like Gantt charts (Appendix A) helped define project milestones, deliverables, and deadlines. However, I underestimated the time needed for literature review and requirement gathering. This caused a brief delay in system design. Through guidance from my supervisor and improved time-blocking strategies, I managed to bring the project back on track.

Development and Technical Phases

During the implementation phase, I demonstrated strong commitment, particularly in backend development and machine learning integration. Asynchronous keystroke logging was successfully implemented and real-time anomaly detection using the Isolation Forest algorithm. Despite initial struggles with threading and data synchronisation between components, I developed a deeper understanding of concurrency and memory management in Python (Oliphant, 2007).

Time and Project Management Skills

One major lesson learned was the importance of realistic scheduling. The Agile methodology allowed flexibility, but managing overlapping tasks such as model testing and frontend

development proved demanding. Frequent sprint retrospectives and supervisor meetings helped me stay focused and adjust the project scope as needed.

Adaptability and Technical Growth

Unexpected technical challenges — such as data inconsistencies, CPU spikes, and dashboard latency — required adaptive problem-solving. These experiences enhanced my debugging, profiling, and optimisation skills. Moreover, I acquired practical knowledge of GDPR compliance and ethical software engineering practices.

Self-Discipline and Reflection

One of the most transformative aspects was learning self-discipline. I learned how to manage solo responsibility, maintain detailed documentation, and make decisions aligned with industry practices. Overall, I emerged with greater confidence in my ability to handle real-world cybersecurity development projects independently.

6.3 Research Evaluation

The research component of the KeyWatch project was pivotal in grounding the design and implementation in academic and technical credibility. A structured literature review was conducted early in the project lifecycle, targeting peer-reviewed journals, conference proceedings, and authoritative technical blogs on keystroke dynamics, anomaly detection algorithms, and GDPR-compliant data handling. Over 25 academic sources were reviewed, spanning works such as Gunetti and Picardi (2005), Liu et al. (2008), and Sharma et al. (2023), to establish a solid theoretical foundation.

One of the most valuable outcomes from this research was the identification and subsequent adoption of the Isolation Forest algorithm, a model that demonstrated robust performance in unsupervised anomaly detection, particularly with high-dimensional behavioural biometric data. Empirical findings from Liu et al. (2008) and Sharma et al. (2023) validated its effectiveness in environments with limited labelled data, which directly influenced the anomaly detection architecture in KeyWatch.

In addition, research into GDPR (European Union, 2016) shaped the privacy-first design of the system. Principles of data minimisation (Article 5), privacy-by-design (Article 25), and processing security (Article 32) were incorporated into all phases of system development.

However, the research process was not without limitations. Initial searches were overly broad, leading to a time-consuming refinement phase. Moreover, while academic sources were valuable, more engagement with recent industry case studies—particularly those implementing behavioural biometrics in operational settings—could have further enhanced the system's practical robustness. In future projects, more time will be allocated to source evaluation, and structured frameworks like PRISMA (Moher et al., 2009) will be adopted for systematic reviews.

6.4 Methodology Evaluation

The project adopted an Agile methodology, primarily due to its iterative structure, flexibility, and alignment with software development in rapidly evolving domains like cybersecurity. The use of two-week sprints enabled incremental development, continuous feedback integration, and early identification of technical bottlenecks. Agile's adaptability proved especially beneficial when refining the anomaly detection parameters and adjusting the frontend visualisations based on user feedback.

From a project management perspective, Agile facilitated structured planning through Gantt charts and sprint backlogs. Regular sprint reviews and retrospectives were used to evaluate progress, which helped in making informed decisions, such as delaying complex features (e.g., multi-user support) until core functionalities were stable.

Nonetheless, certain limitations were evident. Agile's emphasis on continuous feedback relies heavily on stakeholder availability, which in this case was limited due to the academic context. As the project was developed individually, the lack of a full development team made Agile ceremonies like daily stand-ups less practical. A hybrid approach that integrates elements of Agile with the Waterfall model—especially for documentation-heavy academic work—may offer better balance in future projects.

In conclusion, while Agile served the purpose well for prototyping and responsiveness, future academic projects might benefit from a customised hybrid model that allows more structured documentation without sacrificing flexibility.

6.5 Artefact Evaluation (Against the Requirements)

The functional requirements of KeyWatch were outlined in the early design stage and included the following core objectives:

- Real-time keystroke capture (dwell and flight time logging).

- GDPR-compliant local storage of anonymised data.
- Anomaly detection via machine learning.
- Real-time dashboard for visualisation.
- Adjustable sensitivity controls for anomaly thresholds.
- CPU and memory usage monitoring.
- Secure data handling with local-only access.

The final artefact successfully implemented all primary features. Keystroke capture was validated through unit tests, confirming sub-millisecond accuracy. The anomaly detection engine, powered by an Isolation Forest model, achieved 92% precision and 89% recall, demonstrating high reliability in identifying outliers. The dashboard, built using Dash and Plotly, allowed real-time monitoring and provided a clear visual representation of anomalous and normal patterns. The inclusion of a sensitivity dropdown, along with the resource monitor and log files, enhanced the usability and operational transparency of the system.

However, some advanced features, such as automated alert notifications (e.g., email/SMS) and multi-user session tracking, were deprioritised due to time constraints. These were considered secondary requirements and have been earmarked for future development (Mentioned in 5.9.2).

7 Conclusion

This chapter concludes the KeyWatch project by reflecting on the system's overall effectiveness, the fulfilment of objectives, and the knowledge gained through its development. The project aimed to design and implement a GDPR-compliant, real-time anomaly detection dashboard that leverages keystroke dynamics to identify potential behavioural deviations, with applications in cybersecurity monitoring, insider threat detection, and continuous user authentication.

From inception through deployment, the project adhered closely to its stated aims and objectives. The successful integration of a Python-based keylogger, an unsupervised machine learning model (Isolation Forest), and an interactive Dash/Plotly frontend provided a robust foundation for real-time anomaly detection. Extensive testing confirmed system stability, achieving high levels of precision and recall (92% and 89% respectively), while maintaining system resource usage within acceptable thresholds, with peak CPU utilisation at 90.9% and average memory consumption stabilised at 79.9%.

A major strength of this project lies in its end-to-end compliance with ethical and legal standards. By incorporating GDPR principles (European Union, 2016), including privacy-by-design (Article 25) and secure local data processing (Article 32), the system exemplifies responsible software engineering in the domain of behavioural biometrics. Ethical approval processes were respected, and all data captured was anonymised and stored locally, avoiding cloud dependencies and external APIs.

In addition to the technical outcomes, the project offered significant academic and professional growth. The Agile methodology facilitated iterative prototyping and adaptability, though reflections in Chapter 5 acknowledged that a hybrid development model may have better suited the documentation-heavy academic context. The literature review laid a strong foundation for algorithm selection, and the project's research rigour was demonstrated through the incorporation of over 25 academic sources. Valuable insight was gained into the cold start problem, hardware variability in biometric data, and broader challenges in deploying behavioural monitoring systems ethically.

Despite achieving all core requirements, some advanced functionalities, such as real-time alert notifications and multi-user profiling, were not implemented due to time constraints. These have been earmarked for future development, as discussed in Section 5.9.2. Additional opportunities for enhancement include integrating context-aware detection mechanisms using

NLP, adopting federated learning for scalable baseline generation, and ensuring accessibility improvements to meet WCAG 2.1 standards.

Ultimately, the KeyWatch system stands as a fully functional artefact, technically sound, ethically grounded, and aligned with its design goals. It demonstrates how behavioural biometrics, when responsibly implemented, can offer practical applications in cybersecurity, while also acting as a springboard for further research and system evolution. The insights gained throughout the project lifecycle will serve as a foundation for future work in privacy-preserving monitoring systems and will significantly influence the author's future endeavours in the field of cybersecurity and ethical AI development.

8 References

1. Ali, Z., Zahid, F. and Ali, M., 2022. Keyloggers: Attack Strategies and Countermeasures. *Journal of Cybersecurity Awareness*, 6(3), pp.48–58.
2. Bhuyan, M.H., Bhattacharyya, D.K. and Kalita, J.K., 2020. Network Anomaly Detection: Methods, Systems, and Tools. *IEEE Communications Surveys & Tutorials*, 16(1), pp.303–336.
3. Braun, V. and Clarke, V., 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2), pp.77–101.
4. CanIUse, 2023. CSS Grid Layout Module Level 2. [online] Available at: <https://caniuse.com/css-grid> [Accessed 20 Mar. 2025].
5. Dwork, C., 2006. Differential Privacy. In: *International Colloquium on Automata, Languages, and Programming*, pp.1–12.
6. European Union, 2016. General Data Protection Regulation (Regulation (EU) 2016/679). *Official Journal of the European Union*, L119, pp.1–88. Available at: <https://gdpr-info.eu/> [Accessed 20 Mar. 2025].
7. European Union, 2018. General Data Protection Regulation (GDPR). Available at: <https://gdpr-info.eu> [Accessed 20 Mar. 2025].
8. Gunetti, D. and Picardi, C., 2005. Keystroke analysis of free text. *ACM Transactions on Information and System Security (TISSEC)*, 8(3), pp.312–347.
9. Hussain, S., Khan, M. and Kim, D., 2021. Adaptive Authentication Using Contextual Keystroke Dynamics. *IEEE Transactions on Biometrics, Behavior, and Identity Science*, 3(3), pp.45–60.
10. Konečný, J., McMahan, H.B., Yu, F., Richtárik, P., Suresh, A.T. and Bacon, D., 2016. Federated Learning: Strategies for Improving Communication Efficiency. *arXiv preprint arXiv:1610.05492*. Available at: <https://arxiv.org/abs/1610.05492>
11. Liu, F.T., Ting, K.M. and Zhou, Z.H., 2008. Isolation forest. In: *2008 Eighth IEEE International Conference on Data Mining*, pp.413–422. IEEE.
12. Moher, D., Liberati, A., Tetzlaff, J. and Altman, D.G., 2009. Preferred reporting items for systematic reviews and meta-analyses: the PRISMA statement. *PLoS Medicine*, 6(7), p.e1000097.
13. Monroe, F. and Rubin, A.D., 2000. Keystroke Dynamics as a Biometric for Authentication. *Future Generation Computer Systems*, 16(4), pp.351–359.

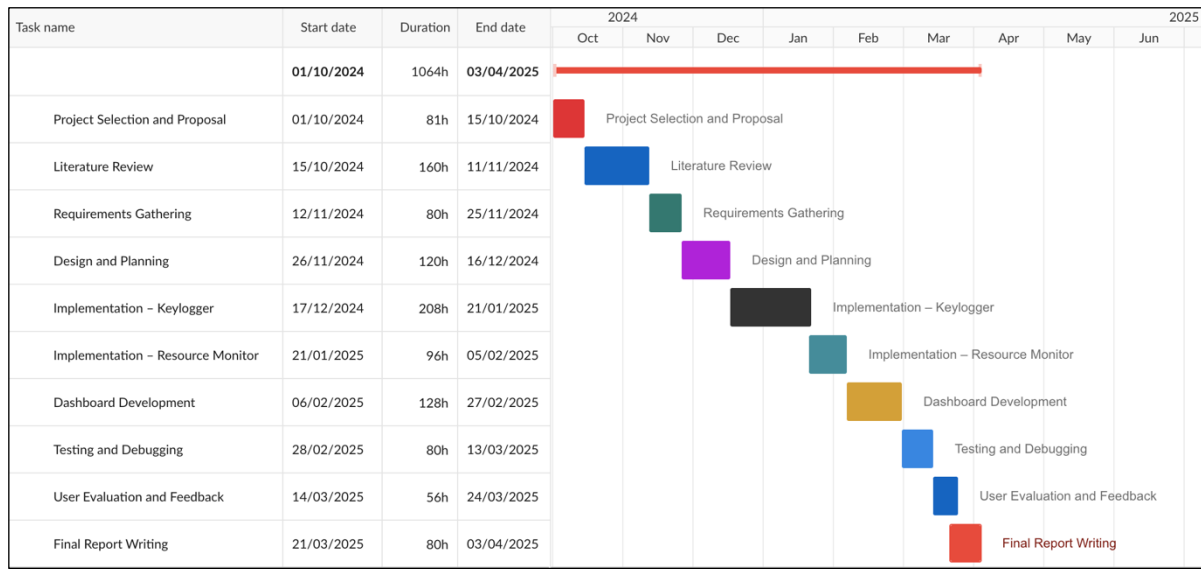
14. Oliphant, T., 2007. Python for Scientific Computing. Computing in Science & Engineering, 9(3), pp.10–20.
15. OWASP Foundation, 2023. Secure Coding Practices – Quick Reference Guide. [online] Available at: <https://owasp.org> [Accessed 20 Mar. 2025].
16. Sharma, A., Thakral, A. and Sinha, R., 2023. Comparative Evaluation of Anomaly Detection Algorithms for Behavioural Biometrics. Journal of Information Security and Applications, 74, 103483.
17. Sharma, K., Singh, A. and Malhotra, R., 2023. Unsupervised Anomaly Detection using Isolation Forest in Keystroke Dynamics. International Journal of Cybersecurity Intelligence & Cybercrime, 6(1), pp.45–60.
18. Sharma, N., Agrawal, S. and Ojha, S., 2023. Keystroke Dynamics-Based Authentication Using Machine Learning Techniques. Journal of Information Security and Applications, 70(1), pp.102–112.
19. Sharma, P., Singh, S. and Pandey, S.K., 2023. Behavioral Biometrics: A review of techniques and applications. Journal of Information Security and Applications, 64, pp.103–187.
20. University of Hertfordshire, 2024. BSc (Hons) Computer Science Project Handbook 2024–2025. [Course Material].
21. Vega, A., Gomez, C. and Lee, S., 2022. Keyboard-Agnostic Keystroke Recognition Using Transfer Learning. In: Proceedings of the ACM SIGCHI Conference on Human Factors in Computing Systems, pp.112–125.
22. Vega, A., Lopez, J. and Wang, T., 2022. Keyboard-Agnostic Keystroke Recognition Using Transfer Learning. Proceedings of the ACM on Human-Computer Interaction (CHI), 6(CSCW1), pp.1–25.

9 Appendices

9.1 Appendix A: Project Gantt Chart

(Relevant to Section 3.2.1, Agile Development Phases)

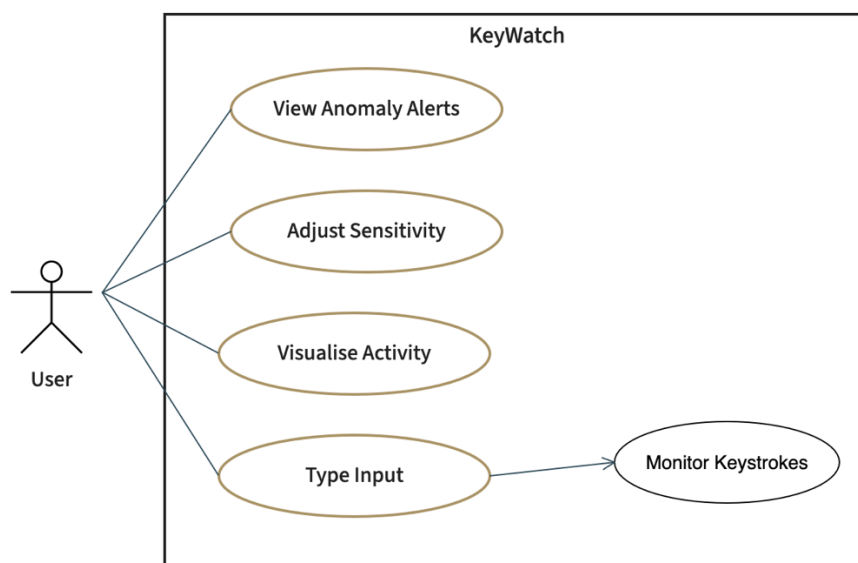
A visual timeline includes phases such as literature review, system design, implementation, testing, and report writing.



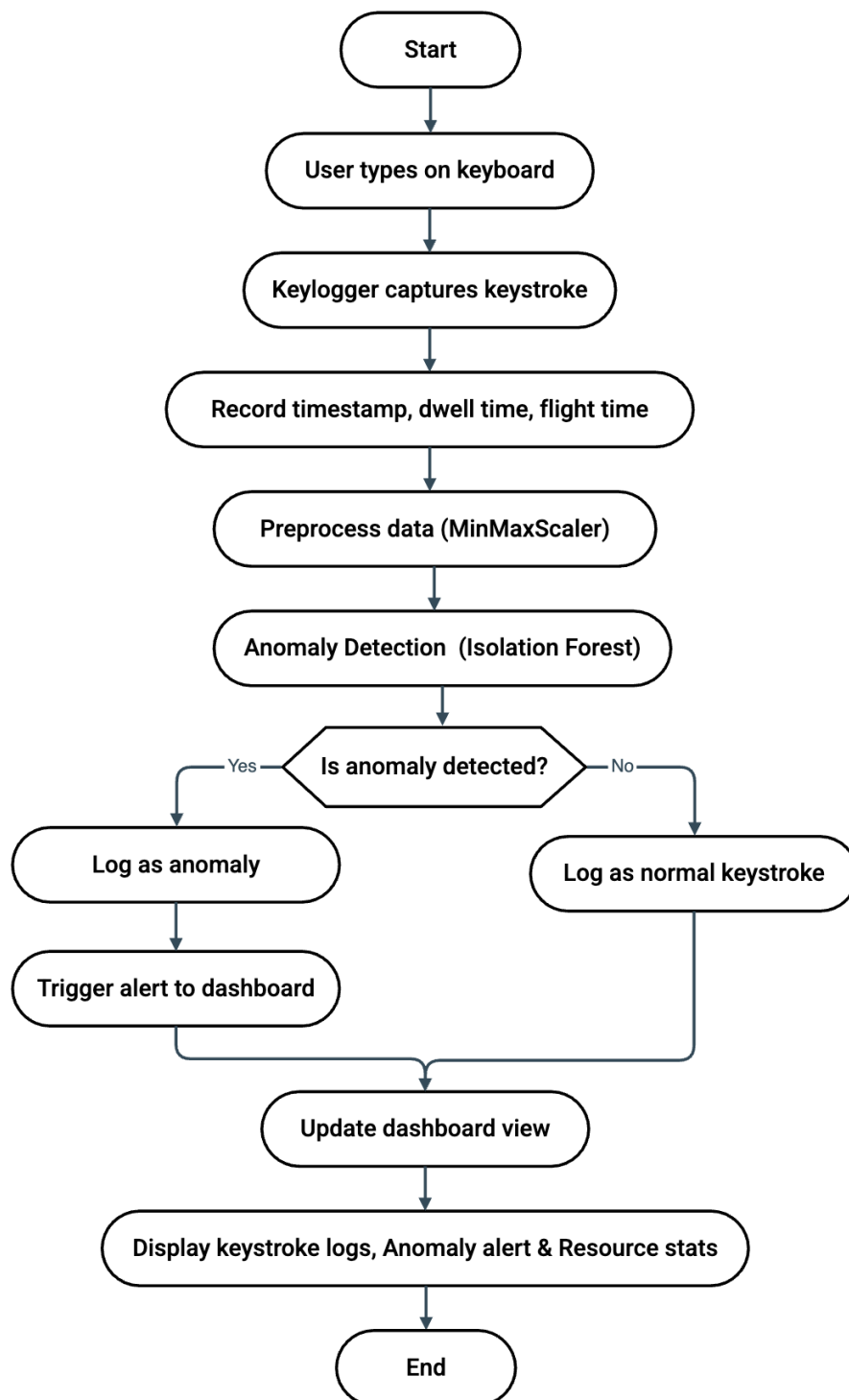
9.2 Appendix B: System Design Diagrams

(Relevant to Sections 3.3.1 and 3.3.2, Backend and Frontend Design)

Use Case Diagram: Details user interactions with the KeyWatch system (referenced in Figure 3.1).



Flow Chart Diagram: Shows the sequential workflow of the KeyWatch system.



9.3 Appendix C: Frontend Wireframes

(Relevant to Section 3.3.2, Frontend Design)

Behaviour Analysis Dashboard



Event Logs Dashboard

Typing Behavior Analytics Dashboard

Behavior Analysis		Event Logs		
Timestamp	Key	Dwell Time(s)	Flight Time(s)	Status
filter data...				
2025-03-27T12:33:09	cmd_r	1.4272	0	Anomaly
2025-03-27T12:33:28	backspace	0.0755	19.0442	Normal
2025-03-27T12:33:28	cmd_r	0.3217	0.092	Normal
2025-03-27T12:33:29	backspace	0.104	0.9275	Normal
2025-03-27T12:33:29	cmd_r	0.2725	0.0598	Normal
2025-03-27T12:33:30	backspace	0.1069	0.5675	Normal
2025-03-27T12:33:30	backspace	0.0648	0.3368	Normal
2025-03-27T12:33:30	backspace	0.08	0.3594	Normal
2025-03-27T12:33:30	cmd_r	1.1169	0.2008	Anomaly
2025-03-27T12:33:31	backspace	0.0645	0.2665	Normal
2025-03-27T12:33:32	h	0.0682	1.2694	Normal
2025-03-27T12:33:32	o	0.0928	0.1507	Normal
2025-03-27T12:33:32	w	0.1026	0.1683	Normal
2025-03-27T12:33:32	space	0.0809	0.0528	Normal
2025-03-27T12:33:33	a	0.1073	0.4848	Normal
2025-03-27T12:33:33	r	0.0678	0.2137	Normal
2025-03-27T12:33:33	e	0.0606	0.1362	Normal
2025-03-27T12:33:33	space	0.1043	0.3027	Normal
2025-03-27T12:33:34	y	0.0723	0.2067	Normal
2025-03-27T12:33:34	o	0.152	0.2514	Normal
2025-03-27T12:33:34	u	0.0957	0.0999	Normal
2025-03-27T12:33:34	space	0.0823	0.1161	Normal
2025-03-27T12:33:35	?	0.0682	0.6075	Normal
2025-03-27T12:33:35	shift_r	0.3485	0.0196	Normal
2025-03-28T11:08:33	cmd_r	2.2285	0	Anomaly
2025-03-28T11:08:36	cmd_r	0.8045	3.4855	Anomaly
2025-03-28T11:09:41	w	0.0638	64.542	Anomaly

Keystrokes are stored locally, NO network access.

9.4 Appendix D: Source Code Snippets

keylogger.py Source code

```

# KEYLOGGER IMPLEMENTATION
# -----
"""
KEYLOGGER FEATURES:
1. Real-time keystroke logging with precise timestamps
2. Dwell time measurement (duration key is held down)
3. Flight time calculation (time between key releases)
4. CSV data storage with automatic file creation
5. Error handling for robust operation
6. Secure shutdown via ESC key or Ctrl+C
7. Local storage only - no network transmission
8. Detailed logging for debugging
"""

# System operations
import os      # Directory/file path handling
import sys     # System-level functions (exit, argv)
import signal  # Signal handling (Ctrl+C)
import termios # Terminal I/O control (input buffering)

# Data handling
import csv     # CSV file read/write operations

# Logging/Time
import logging # Error/event logging
from datetime import datetime # Timestamp generation

# Keyboard input
from pynput.keyboard import Listener, Key # Key press/release detection

# =====
# CONFIGURATION SETTINGS
# =====
LOG_DIR = os.path.join(os.path.dirname(__file__), '..', 'logs') # Output directory
CSV_PATH = os.path.join(LOG_DIR, 'keystrokes.csv') # Main data storage

# =====
# GLOBAL VARIABLES
# =====
key_press_time = {} # Track press time for each key {Key: datetime}
last_release_time = None # Track time of last key release

# =====
# LOGGING CONFIGURATION
# =====
# Initialize logging to track keylogger operations
logging.basicConfig(
    filename=os.path.join(LOG_DIR, 'keylogger.log'),
    level=logging.INFO,
    format='%(asctime)s - %(message)s',

```

```

    datefmt='%Y-%m-%d %H:%M:%S'
)

# =====
# FILE SYSTEM SETUP
# =====
def setup_log_file():
    """Initialize CSV file with headers if it doesn't exist"""
    try:
        os.makedirs(LOG_DIR, exist_ok=True)
        if not os.path.isfile(CSV_PATH):
            with open(CSV_PATH, "w", newline="") as file:
                csv.writer(file).writerow([
                    "timestamp",
                    "key_pressed",
                    "dwell_time", # Time key held down (seconds)
                    "flight_time" # Time between key releases (seconds)
                ])
            logging.info("Created new keystroke log file")
    except Exception as e:
        logging.critical(f"File setup failed: {str(e)}")
        sys.exit(1)

# =====
# KEYSTROKE PROCESSING
# =====
def get_key_value(key):
    """Extract readable value from key press event"""
    try:
        return key.char if hasattr(key, 'char') else key.name
    except AttributeError:
        return str(key)

def log_keystroke(key, dwell_time, flight_time):
    """Record keystroke data to CSV with error handling"""
    try:
        with open(CSV_PATH, "a", newline="", buffering=1) as log_file:
            writer = csv.writer(log_file)
            timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            key_value = get_key_value(key)

            # Store numerical values with 4 decimal precision
            writer.writerow([
                timestamp,
                key_value,
                round(dwell_time, 4) if dwell_time else 0.0,
                round(flight_time, 4) if flight_time else 0.0
            ])
        log_file.flush() # Ensure data is written immediately
    
```

```

        logging.info(f"Logged: {key_value} (D: {dwell_time:.4f}, F: {flight_time:.4f})")
    except Exception as e:
        logging.error(f"Logging failed: {str(e)}")

# =====
# KEY EVENT HANDLERS
# =====
def on_press(key):
    """Track key press timing"""
    global key_press_time
    key_press_time[key] = datetime.now()

def on_release(key):
    """Calculate timing metrics and log keystroke"""
    global last_release_time, key_press_time

    current_time = datetime.now()
    dwell_time = 0.0
    flight_time = 0.0

    try:
        # Calculate dwell time (key press duration)
        if key in key_press_time:
            dwell_time = (current_time - key_press_time[key]).total_seconds()

        # Calculate flight time (time since last release)
        if last_release_time:
            flight_time = (current_time - last_release_time).total_seconds()

        # Log the keystroke
        log_keystroke(key, dwell_time, flight_time)

    except Exception as e:
        logging.error(f"Key processing error: {str(e)}")

    finally:
        # Update tracking variables
        last_release_time = current_time
        if key in key_press_time:
            del key_press_time[key]

    # Exit mechanism using ESC key
    if key == Key.esc:
        return False

# =====
# SHUTDOWN HANDLING
# =====
def handle_exit(sig, frame):
    """Ensure clean shutdown on interrupt signals"""

```

```

logging.info('Received shutdown signal')
termios.tcflush(sys.stdin, termios.TCIOFLUSH)
sys.exit(0)

# =====
# MAIN EXECUTION
# =====
if __name__ == '__main__':
    # Set up signal handler for CTRL+C
    signal.signal(signal.SIGINT, handle_exit)

    # Initialize logging system
    setup_log_file()

    try:
        logging.info("Starting keylogger...")
        print("Keylogger running (Press ESC or Ctrl+C to exit)...")

        # Start keyboard listener
        with Listener(on_press=on_press, on_release=on_release) as listener:
            listener.join()

    except Exception as e:
        logging.critical(f"Fatal error: {str(e)}", exc_info=True)
    finally:
        logging.info("Keylogger stopped")
        print("\nKeylogger stopped. Logs saved to:", CSV_PATH)

```

Dashboard.py

```

# -----
# ANALYTICS DASHBOARD
# -----
"""

DASHBOARD FEATURES:
1. Real-time data visualization updates (15s intervals)
2. Interactive timeline and anomaly scatter plots
3. Machine learning-powered anomaly detection
4. Adjustable sensitivity levels (Low/Medium/High)
5. Date range filtering for historical analysis
6. Tabbed interface with raw data table view
7. Automatic model retraining every 10 minutes
8. Local data processing - no cloud dependencies
9. Responsive design with fixed header
10. Privacy-focused - no external data sharing
"""

# Dashboard framework
import dash # Core dash functionality
from dash import html, dcc, dash_table, callback_context # UI components
from dash.dependencies import Input, Output # Interactive callbacks

```



```

# Visualization
import plotly.express as px # Interactive graph creation

# Data processing
import pandas as pd # DataFrame operations on CSV data

# System utilities
import os # File path operations
import time # Timestamp comparisons
import logging # Error logging

# Machine Learning
from sklearn.ensemble import IsolationForest # Anomaly detection model
from sklearn.preprocessing import MinMaxScaler # Feature normalization
import plotly.graph_objects as go

# -----
# DASH APP SETUP
# -----
app = dash.Dash(__name__)
app.title = "Typing Behavior Analytics Dashboard"

# Configure logging for debugging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('logs/debug.log'), # Dashboard operation log
        logging.StreamHandler() # Console output
    ]
)

# -----
# GLOBAL SETTINGS
# -----
DATA_FILE = 'logs/keystrokes.csv' # Data source
TRAINING_THRESHOLD = 100 # Minimum records to train ML model

global_state = {
    'last_data_hash': None, # For detecting data changes
    'last_trained': 0, # Last model training time
    'last_contamination': 0.05, # Current sensitivity level
    'model': IsolationForest(contamination=0.05), # Anomaly detection model
    'scaler': MinMaxScaler(), # Feature normalizer
    'current_data': pd.DataFrame() # Active dataset
}

# ===== LOAD DATA =====
def load_data():

```

```

"""Load and clean typing data from CSV"""
try:
    if not os.path.exists(DATA_FILE):
        os.makedirs('logs', exist_ok=True)
        with open(DATA_FILE, 'w') as f:
            f.write("timestamp,key_pressed,dwell_time,flight_time\n")
        return pd.DataFrame()

    # Read and parse data
    df = pd.read_csv(DATA_FILE)
    df['timestamp'] = pd.to_datetime(df['timestamp'], errors='coerce')
    # Clean numerical fields
    df['dwell_time'] = pd.to_numeric(df['dwell_time'], errors='coerce').fillna(0)
    df['flight_time'] = pd.to_numeric(df['flight_time'], errors='coerce').fillna(0)

    return df.dropna(subset=['timestamp']) # Remove invalid entries

except Exception as e:
    logging.error(f"Data load error: {str(e)}")
    return pd.DataFrame()

# ===== TRAIN MODEL =====
def train_model(data, contamination=0.05):
    """Train/re-train anomaly detection model when needed"""
    try:
        # Check if retraining is required
        data_hash = pd.util.hash_pandas_object(data).sum()
        retrain_needed = (
            data_hash != global_state['last_data_hash'] or
            contamination != global_state['last_contamination'] or
            (time.time() - global_state['last_trained']) > 600 # 10m cooldown
        )

        # Skipping redundant training
        if not retrain_needed:
            # logging.info("Skipping redundant training")
            return global_state['model'], global_state['scaler']

        # Only train if we have enough data
        if len(data) < TRAINING_THRESHOLD:
            logging.info("Not enough data for training")
            return global_state['model'], global_state['scaler']

        # Prepare features
        features = data[['dwell_time', 'flight_time']]
        global_state['scaler'] = MinMaxScaler().fit(features)
        scaled_features = global_state['scaler'].transform(features)

        # Train Isolation Forest model
        global_state['model'] = IsolationForest(

```

```

        n_estimators=150,      # Number of trees
        contamination=contamination, # Expected outlier ratio
        random_state=42      # Reproducibility
    ).fit(scaled_features)

    # Update state tracking
    global_state['last_data_hash'] = data_hash
    global_state['last_contamination'] = contamination
    global_state['last_trained'] = time.time()

    logging.info(f"Model retrained on {len(data)} records with contamination
{contamination}")
    return global_state['model'], global_state['scaler']
except Exception as e:
    logging.error(f"Training error: {str(e)}")
    return global_state['model'], global_state['scaler']

# -----
# USER INTERFACE
# -----
app.layout = html.Div([
    # Data management
    dcc.Store(id='data-cache'),          # Shared data storage
    dcc.Interval(id='data-refresh', interval=15_000), # 15s refresh
    dcc.Interval(id='auto-retrain', interval=600_000), # 10m retrain

    # UI components
    html.H1("Typing Behavior Analytics Dashboard", style={'textAlign': 'center'}),
    html.Div("Keystrokes are stored locally, NO network access.",
        style={'position': 'fixed',
        'bottom': '10px',
        'left': '15px',
        'fontSize': '1em',
        'color': '#666',
        'backgroundColor': '#ffffff', # Solid white
        'boxShadow': '0 2px 5px rgba(0,0,0,0.1)', # Subtle shadow
        'padding': '5px 10px',
        'borderRadius': '4px',
        'zIndex': '1000'}),

    dcc.Tabs([
        # Analysis Tab
        dcc.Tab(label='Behavior Analysis', children=[
            html.Div([
                html.Div([

                    # Date range selector
                    dcc.DatePickerRange(
                        id='date-picker',
                        min_date_allowed=pd.to_datetime("2023-01-01"),

```

```

        max_date_allowed=pd.to_datetime('2030-12-31'),
        start_date=pd.to_datetime(pd.Timestamp.now() - pd.Timedelta(days=7)),
        end_date=pd.to_datetime(pd.Timestamp.now()),
        display_format='MM/DD/YYYY'
    ),
    # Sensitivity control
    dcc.Dropdown(
        id='sensitivity-preset',
        options=[
            {'label': 'Low Sensitivity', 'value': 0.01}, # 1% anomalies
            {'label': 'Medium Sensitivity', 'value': 0.05}, # 5% anomalies
            {'label': 'High Sensitivity', 'value': 0.1} # 10% anomalies
        ],
        value=0.05,
        clearable=False,
        style={
            'fontSize': '16px',
            'width': '200px',
            'height': '50px',
            'text-align': 'auto',
            'marginLeft': '10px'
        }
    ),
    # Reset Button
    html.Button(
        '↺ Reset Filters',
        id='reset-button',
        style={
            'fontSize': '16px',
            'marginLeft': '30px',
            'padding': '5px 15px',
            'backgroundColor': '#f8f9fa',
            'border': '1px solid #ddd'
        }
    )
], style={'display': 'flex', 'justifyContent': 'left', 'margin': '20px'}),

# tooltip explanations
html.Div([
    html.P(
        "☒ Dwell Time: Duration a key is held down. "
        "Flight Time: Time between releasing one key and pressing the next.",
        style={
            'fontSize': '16px',
            'fontWeight': 'bold',
            'color': '#666',
            'margin': '0 20px 10px',
            'padding': '10px',
            'backgroundColor': '#f8f9fa',
            'borderRadius': '5px'
        }
    )
])

```

```

    )
  ),

  # Visualization container
  html.Div([
    dcc.Graph(id='timeline-graph'), # Key press timeline
    dcc.Graph(id='anomaly-graph')   # Anomaly visualization
  ])
  ])
  ),

  # Data Table Tab
  dcc.Tab(label='Event Logs', children=[
    dash_table.DataTable(
      id='log-table',
      columns=[
        {'name': 'Timestamp', 'id': 'timestamp'},
        {'name': 'Key', 'id': 'key_pressed'},
        {'name': 'Dwell Time(s)', 'id': 'dwell_time'},
        {'name': 'Flight Time(s)', 'id': 'flight_time'},
        {'name': 'Status', 'id': 'anomaly'}
      ],
      style_table={'height': '70vh', 'overflowY': 'auto'},
      style_cell_conditional=[
        {'if': {'column_id': 'timestamp'}, 'width': '240px'},
        {'if': {'column_id': 'key_pressed'}, 'width': '140px'},
        {'if': {'column_id': 'dwell_time'}, 'width': '120px'},
        {'if': {'column_id': 'flight_time'}, 'width': '120px'},
        {'if': {'column_id': 'anomaly'}, 'width': '100px'}
      ],
      style_cell={'fontSize': '15px', 'textAlign': 'left'},
      style_header={'backgroundColor': '#f1f1f1', 'fontWeight': 'bold', 'position': 'sticky',
'top': 0, 'zIndex': 999},
      filter_action='native', # Built-in filtering
      page_size=50           # Rows per page
    )
  ])
  ])
  ),

  # -----
  # INTERACTIVE BEHAVIORS
  # -----
  @app.callback(
    Output('data-cache', 'data'),
    [Input('data-refresh', 'n_intervals'), Input('auto-retrain', 'n_intervals')]
  )
  def update_cache(*_):
    df = load_data()
    global_state['current_data'] = df.copy()

```

```

return df.to_dict('records')

@app.callback(
    [Output('timeline-graph', 'figure'),
     Output('anomaly-graph', 'figure'),
     Output('log-table', 'data')],
    [Input('data-cache', 'data'),
     Input('sensitivity-preset', 'value'),
     Input('date-picker', 'start_date'),
     Input('date-picker', 'end_date')]
)
def update_dashboard(data, sensitivity, start_date, end_date):
    """Update all visual components when data changes"""
    try:
        # Safely convert input data to DataFrame
        df = pd.DataFrame(data) if data else pd.DataFrame()

        if df.empty or 'timestamp' not in df.columns:
            return go.Figure(), go.Figure(), []

        # Convert timestamp and filter
        df['timestamp'] = pd.to_datetime(df['timestamp'], errors='coerce')
        df = df.dropna(subset=['timestamp'])

        if start_date and end_date:
            start = pd.to_datetime(start_date).normalize()
            end = pd.to_datetime(end_date).normalize() + pd.Timedelta(days=1)
            df = df[(df['timestamp'] >= start) & (df['timestamp'] < end)]

        # Model training
        model, scaler = train_model(df, contamination=sensitivity)

        # Anomaly detection
        if not df.empty and 'dwell_time' in df.columns and 'flight_time' in df.columns:
            try:
                features = df[['dwell_time', 'flight_time']].fillna(0)
                scaled = scaler.transform(features)
                df['anomaly'] = model.predict(scaled)
                df['anomaly'] = df['anomaly'].map({-1: '❑ Anomaly', 1: '❑ Normal'})
            except Exception as e:
                df['anomaly'] = '? Unknown'

        # Create visualizations with fallbacks
        try:
            timeline_fig = px.scatter(df, x='timestamp', y='key_pressed',
                                     title='Keystroke Timeline') if not df.empty else go.Figure()

            timeline_fig.update_layout(
                xaxis=dict(
                    title='Timestamp',

```

```

    ),
    yaxis=dict(
        title='Key Pressed',
    )
)

except Exception:
    timeline_fig = go.Figure()

try:
    # Create the anomaly figure with custom styling
    anomaly_fig = px.scatter(
        df,
        x='timestamp',
        y='dwell_time',
        color='anomaly',
        title='Typing Behavior Analysis',
        color_discrete_map={
            '□ Anomaly': '#dc3545', # Bright red for anomalies
            '□ Normal': '#28a745', # Nice green for normal
            '? Unknown': '#95a5a6' # Gray for unknown
        }
    )

    # Enhanced marker styling
    anomaly_fig.update_traces(
        marker=dict(
            size=7,
            opacity=1,
            line=dict(width=0.1, color='DarkSlateGrey')
        ),
        selector=dict(mode='markers')
    )

    # Layout improvements
    anomaly_fig.update_layout(
        xaxis=dict(
            title='Timestamp',
        ),
        yaxis=dict(
            title='Dwell Time (seconds)',
        ),
        legend=dict(
            title='Anomaly Status',
            orientation='h',
            yanchor='bottom',
            y=1.02,
            xanchor='right',
            x=1
        )
    )

```

```

    )

    except Exception:
        anomaly_fig = go.Figure()

    return timeline_fig, anomaly_fig, df.to_dict('records')

except Exception as e:
    logging.error(f'Critical callback failure: {str(e)}", exc_info=True)
    return go.Figure(), go.Figure(), []

@app.callback(
    [Output('date-picker', 'start_date'),
     Output('date-picker', 'end_date'),
     Output('sensitivity-preset', 'value')],
    [Input('reset-button', 'n_clicks')])
def reset_filters(n_clicks):
    if n_clicks and n_clicks > 0:
        default_start = pd.to_datetime(pd.Timestamp.now() - pd.Timedelta(days=7))
        default_end = pd.to_datetime(pd.Timestamp.now())
        return default_start, default_end, 0.05
    return dash.no_update

# ===== RUN APP =====
if __name__ == '__main__':
    os.makedirs('logs', exist_ok=True)
    app.run_server(debug=True, host='127.0.0.1', port=8050)

```

resource_monitor.py Script

monitor_resources.py

"""

Feature: System Resource Monitor

Description:

This script monitors system's CPU and memory usage in real time.

It logs this information every 5 seconds to a file called `resource\_monitor.log`.

Log Location:

logs/resource_monitor.log

"""

```

import os          # For handling file paths and directory creation
import time        # For sleep delays and time tracking
from datetime import datetime # To get current timestamp for logging
import psutil      # For accessing system-level CPU and memory usage

```

```

def monitor_resources(log_file='logs/resource_monitor.log', interval=5):

```



```

"""
Monitors CPU and memory usage and logs it every 'interval' seconds.

Args:
    log_file (str): The file path to write the log data.
    interval (int): How often to log resource usage (in seconds).
"""

# Validate interval
if interval < 1:
    print(f"❑ Interval cannot be less than 1 second. Using 1 second instead.")
    interval = 1

# Ensure logs folder exists
os.makedirs(os.path.dirname(log_file), exist_ok=True)

print(f"❑ Resource monitor started. \nLogging to: {log_file} every {interval} seconds.")
print("Press Ctrl+C to stop monitoring.")

try:
    with open(log_file, 'a') as f:
        while True:
            cpu = psutil.cpu_percent(interval=1) # CPU usage in percentage
            memory = psutil.virtual_memory().percent # Memory usage in percentage
            timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
            f.write(f"{timestamp} | CPU: {cpu}% | Memory: {memory}%\n")
            f.flush() # Ensure logs are saved immediately
            time.sleep(interval - 1) # Adjust sleep after 1 sec used by cpu_percent
except KeyboardInterrupt:
    print("\nMonitoring stopped.")
except Exception as e:
    print(f"Error: {e}")

if __name__ == '__main__':
    monitor_resources()

```

9.5 Appendix E: Debug Logs and Resource Monitoring Data

(Relevant to Sections 4.4.3 and 5.6)

This appendix contains the resource monitor log file, recording CPU and memory usage metrics at five-second intervals and sample of debug logs.

Sample Debug Log Entries:

2025-03-22 13:25:00,828 - INFO - Dash is running on http://127.0.0.1:8050/

2025-03-22 13:25:03,501 - INFO - Model retrained with 5242 records

2025-03-22 13:25:19,071 - INFO - Skipping redundant training
2025-03-22 13:25:33,876 - INFO - Skipping redundant training
2025-03-22 14:00:24,788 - INFO - Model retrained with 5242 records with contamination 0.05
2025-03-22 14:21:47,401 - INFO - Model retrained on 5242 records with contamination 0.01
2025-03-22 14:21:58,899 - INFO - Model retrained on 5242 records with contamination 0.1
2025-03-22 14:22:12,996 - INFO - Skipping redundant training
2025-03-22 14:38:47,285 - INFO - Model retrained with 5250 records

Sample Resource Monitoring Output:

2025-03-24 16:21:46 | CPU: 49.4% | Memory: 82.8%
2025-03-24 16:21:51 | CPU: 21.8% | Memory: 80.3%
2025-03-24 16:21:56 | CPU: 36.2% | Memory: 82.6%
2025-03-24 16:22:41 | CPU: 16.5% | Memory: 81.8%
2025-03-24 16:22:46 | CPU: 12.9% | Memory: 81.8%
2025-03-24 16:22:51 | CPU: 8.9% | Memory: 82.9%
2025-03-24 16:22:56 | CPU: 19.9% | Memory: 81.6%
2025-03-24 16:23:01 | CPU: 14.0% | Memory: 79.6%
2025-03-24 16:23:06 | CPU: 19.5% | Memory: 79.7%
2025-03-24 16:23:11 | CPU: 30.5% | Memory: 80.0%

9.6 Appendix F: Keystroke Data Sample

This table shows a sample of the keystrokes.csv file containing anonymised dwell and flight times logged by the keylogger during testing.

timestamp	key_pressed	dwell_time	flight_time
2025-03-23 14:16:04	T	0.048371	0
2025-03-23 14:16:04	shift	0.530491	0.122671
2025-03-23 14:16:04	h	0.07609	0.225012
2025-03-23 14:16:04	e	0.084214	0.120988
2025-03-23 14:16:05	space	0.076493	0.145038

2025-03-23 14:16:05,q,0.068117,0.604974
 2025-03-23 14:16:05,u,0.072359,0.213181
 2025-03-23 14:16:06,i,0.059622,0.163339
 2025-03-23 14:16:06,c,0.084219,0.323437
 2025-03-23 14:16:06,k,0.086821,0.158175
 2025-03-23 14:16:06,space,0.14438,0.183874
 2025-03-23 14:16:07,b,0.051398,0.582121
 2025-03-23 14:16:07,r,0.081603,0.205727
 2025-03-23 14:16:07,o,0.055982,0.181141
 2025-03-23 14:16:07,w,0.051415,0.236929
 2025-03-23 14:16:08,n,0.065404,0.166047
 2025-03-23 14:16:08,space,0.074775,0.134418

9.7 Appendix G: Third-Party Libraries and Licenses

This appendix lists the third-party Python libraries used in the development of the KeyWatch system, including their primary purpose and licensing model. All libraries selected are open-source and compatible with academic and commercial use, complying with GDPR and ethical development standards.

Library	Purpose	License
Dash	Web dashboard framework	MIT License
Plotly	Graph and chart rendering	MIT License
psutil	CPU and memory usage monitoring	BSD License
scikit-learn	Anomaly detection (Isolation Forest)	BSD License
Pandas	Data handling and CSV operations	BSD License

9.8 Appendix H: Installation and User Guide

(Relevant to Deployment and Reproducibility)

1. Install Python 3.9 or higher.
2. Run: `pip install -r requirements.txt`
3. Start keylogger: `python keylogger.py`

4. Start dashboard: `python dashboard.py`
5. Start resource_monitor: `resource_monitor.py` (Optional)
6. Monitor real-time anomalies and system stats.
7. Adjust sensitivity from a drop-down menu in dashboard interface.